

# SECO: Multi-Satellite Edge Computing Enabled Wide-Area and Real-Time Earth Observation Missions

Zhiwei Zhai, Liekang Zeng, Tao Ouyang, Shuai Yu, Qianyi Huang and Xu Chen

School of Computer Science and Engineering, Sun Yat-sen University, China

Email: {zhaizhw3, zenglk3, ouyt9}@mail2.sysu.edu.cn, {yushuai, huangqy89, chenxu35}@mail.sysu.edu.cn

**Abstract**—Rapid advances in low Earth orbit (LEO) satellite technology and satellite edge computing (SEC) have facilitated a key role for LEO satellites in enhanced Earth observation missions (EOM). These missions (e.g., remote object detection) typically require multi-satellite cooperative observations of a large region of interest (RoI) area, as well as the observation image routing and computation processing, enabling accurate and real-time responsiveness. However, optimizing the resources of LEO satellite networks is nontrivial in the presence of its dynamic and heterogeneous properties. To this end, we propose SECO, a SEC-enabled framework that jointly optimizes multi-satellite observation scheduling, routing and computation node selection for enhanced EOM. Specifically, in the observation phase, we leverage the orbital motion and the rotatable onboard cameras of satellites, and propose a distributed game-based scheduling strategy to minimize the overall size of captured images while ensuring full (observation) coverage. In the sequent routing and computation phase, we first adopt image splitting technology to achieve parallel transmission and computation. Then, we propose an efficient iterative algorithm to jointly optimize image splitting, routing and computation node selection for each captured image. On this basis, we propose a theoretically guaranteed system-wide greedy-based strategy to reduce the total time cost (i.e., transmission, computation and queuing delay) over simultaneous processing for multiple images. Extensive experiments based on real-world datasets demonstrate that SECO can achieve up to a 60.7% reduction in overall time cost compared to baselines.

## I. INTRODUCTION

Nowadays, the rapid advancements in low Earth orbit (LEO) satellite technology have significantly enhanced the capacity of LEO satellites in space, which are equipped with high-resolution cameras, communication modules and sophisticated processors [1]. This technological configuration enables LEO satellites to perform timely near-data processing at the edge of LEO satellite networks, thereby giving birth to the concept of satellite edge computing (SEC) [2]. Furthermore, LEO satellites with inter-satellite links (ISLs) [3] can connect with their adjacent satellites, thereby enhancing the utilization of inter-satellite resources to boost service performance. Serving as the centerpiece of SEC, LEO satellites have been extensively utilized in various Earth observation missions (EOM) [4]–[6]. These missions encompass a wide range of critical applications, including carbon emission monitoring [7] and emergency response to diverse natural weather events [8].

The EOM typically involves: i) observation phase, i.e., making observations of a region of interest (RoI), and ii) process-

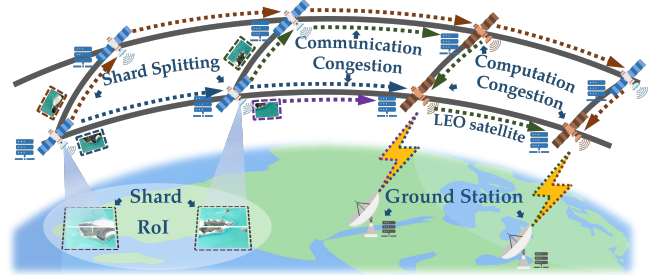


Fig. 1: Multi-satellite cooperative observation, routing and computation process of SECO.

ing phase, i.e., computationally processing the captured RoI images and routing the results to the designated ground station. In terms of the observation phase, major work utilizes a single satellite to handle the observation of a RoI [6], [9], [10]. Nonetheless, in scenarios like earthquake warnings, observing a large RoI area is crucial for a more accurate and satisfactory response [5], [7], [11]. Therefore, as shown in Fig. 1, the multi-satellite cooperative observation becomes indispensable due to the limited observation range of a single satellite. Regarding the processing phase, EOM is often built upon advanced functionalities such as deep neural network (DNN) inference [12] or multi-stage image compression (MIC) [13], necessitating high efficiency to ensure real-time response for time-sensitive missions [14]. Given these attributes, we term this novel EOM paradigm as wide-area and real-time EOM.

To achieve wide-area and real-time EOM, several challenges must be addressed. First, as depicted in Fig. 2, cross-observation areas by ascending and descending satellites divide the RoI into shards. Notably, for the given boundary shard, the satellite capture properties lead to divergent capture sizes for ascending and descending satellites [5], [15]. For instance, for boundary shard *A* in Fig. 2, an ascending satellite only needs to capture the shadow area during its motion direction, while a descending satellite needs to capture the entire segment. Moreover, the observation regions for each satellite are time-varying, due to its dynamic property, i.e., orbital motion and the rotatable onboard camera. In this way, efficient sustained cooperation for multiple satellites in the observation phase is nontrivial. Second, the computation of each shard in SEC involves tasks such as DNN inference or MIC that are computationally intensive. To fully leverage the resources within LEO satellite networks, these tasks can be divided into subtasks and assigned to satellite nodes [16]. However, the inherent dependencies within the computation workflow pose a challenge [17]. Additionally, the heterogeneous nature of satellite hardware resources further compli-

This work was supported in part by the National Science Foundation of China (No. U20A20159, No. 62272122); Guangdong Basic and Applied Basic Research Foundation (No. 2021B151520008, No. 2022B1515120002); Guangzhou Basic and Applied Basic Research Program (No. 2024A04J6367); the Guangdong Science and Technology Planning Project (No. 2018B030322004). Corresponding Author: Xu Chen.

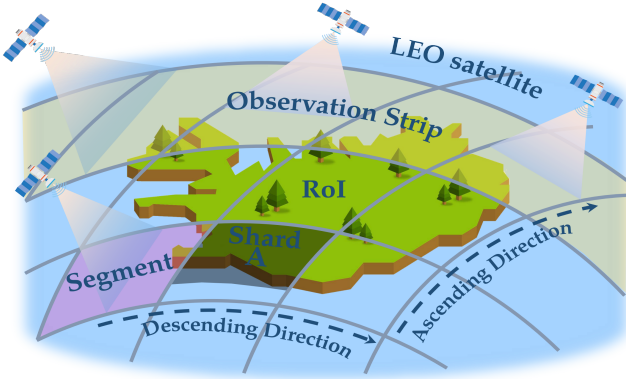


Fig. 2: Multi-satellite cooperative observation for a large RoI. captures the embedding of subtasks within the LEO satellite networks. Finally, as shown in Fig. 1, since the satellite is in motion to capture shards, different satellites might capture shards simultaneously, potentially causing congestion (*i.e.*, queuing delays) on computation nodes and transmission links [18] when they are transmitting and computing. Hence, the complexity of optimizing the processing phase is significantly escalated while considering these queuing delays.

To address the above challenges, we propose SECO, a SEC-enabled framework for jointly optimizing multi-satellite observation scheduling, routing and computation node selection to facilitate wide-area and real-time EOM. In the observation phase, we strategically employ game theory to enable distributed inter-satellite interactions, thus achieving an efficient multi-satellite observation scheduling for a large RoI area. This approach considers the capture and dynamic properties of satellites, enabling the reduction of the overall captured image size while ensuring full RoI coverage. In the processing phase, to tackle the challenge presented by the dependent property in computation tasks (*i.e.*, DNN inference or MIC) and clarify the mapping of processing for each shard in LEO satellite networks, we model the LEO satellite networks as a layered graph to represent the routing and computation process for embedding computation tasks. We theoretically prove that this model scheme yields an integer optimal solution under linear relaxation, thus enabling efficient optimization of the processing for each shard while considering the queuing delay. Additionally, we augment processing efficiency via shard splitting, which allows for parallel transmission and computation. Supporting this, we design an iterative algorithm for jointly shard splitting, routing and computation node selection for each shard. Finally, due to the satellites capturing shards in motion, when multiple shards are captured simultaneously, a theoretically guaranteed system-wide greedy-based strategy is proposed to optimize the time cost at a system level.

In summary, the main contributions are as follows:

- We propose SECO, a comprehensive framework that optimizes observation scheduling, routing and computation node selection for wide-area and real-time EOM, which has great potential in boosting the real-time responses (*i.e.*, to enable efficient DNN inference or MIC, *etc.*).
- We promote a distributed observation scheduling strategy by resorting to the potential game theory as the algorithm

design tool based on the orbital motion and rotatable camera of each satellite. To our knowledge, it is the first strategy considering multi-satellite observations for the large RoI area and fine-grained boundary-side scheduling.

- In SECO, we employ shard splitting to facilitate parallel transmission and computation, thereby reducing time costs. Then an efficient iterative algorithm is proposed to jointly determine the optimal shard splitting, routing and computation node selection for each shard. Further, a system-wide greedy-based strategy is designed to optimize the time cost at the system level, providing performance guarantees through approximation ratio analysis.
- We conduct extensive experiments based on real-world datasets, showing that SECO can achieve up to a 60.7% time cost reduction compared to other baselines.

## II. SYSTEM MODEL

As shown in Fig. 1, consider a multi-satellite cooperative system for joint observation, routing and computation for a large RoI area consisting of a set  $\mathcal{N} = \{1, 2, \dots, N\}$  of LEO satellites and a set  $\mathcal{B} = \{1, 2, \dots, B\}$  of ground stations. According to the dynamic and predictable properties of LEO satellite movement orbits, a given time horizon  $\mathcal{T} = [t_0, t_T]$  can be divided into  $T$  consecutive time slots  $\{\tau_1, \dots, \tau_l, \dots, \tau_T\}$ , where  $\tau_l = [t_{l-1}, t_l]$  and the network topology (*i.e.*, the connectivity among the satellites) is fixed during timeslot  $\tau_l$ , but can be varying in different time slots [10]. Let  $\mathcal{G}_u^{\tau_l} = \{\mathcal{V}_u^{\tau_l}, \mathcal{E}_u^{\tau_l}\}$  denote the topology of LEO satellite networks at time slot  $\tau_l$ , where  $\mathcal{V}_u^{\tau_l} = \mathcal{N} \cup \mathcal{B}$  and  $\mathcal{E}_u^{\tau_l}$  is the set of communication links in LEO satellite networks.

### A. Observation Scheduling

For RoI observation, as shown in Fig. 2, a satellite on each orbit can observe a stripe on the Earth's surface when its camera is pointing vertically downward. Particularly, when the camera angle is rotated within the allowed range, a different stripe can be directed. A stripe is composed of segments, corresponding to its intersections with the stripes in the opposite direction, and each portion of a RoI included in a segment is named **shard**. Denote that a set  $\mathcal{N}'$  ( $\mathcal{N}' \subset \mathcal{N}$ ,  $|\mathcal{N}'| = V$ ) of satellites approaching the observation RoI are used as observation satellites. Moreover, consider a RoI consisting of  $M$  shards, where the shard set  $\mathcal{M} = \{1, 2, \dots, M\}$  need to be assigned to the satellites in  $\mathcal{N}'$  for observing. Then the captured shards would be split into subshards and subsequently routed through satellites to reach the destination ground station, with the computation occurring during this routing process.

In existing implementations, observations of a large RoI area are made by satellites following their own trajectory over a complete observation stripe at the current camera angle [5]. However, when the ascending and descending satellites are observed together, this leads to a large redundancy of observations, resulting in unnecessary duplication of shard transmissions and computations. Since the satellite camera angle can be rotated [15], multiple satellite observations of shards in an RoI can be scheduled to optimize coverage and to reduce the duplication of observations over the same shard.

For each satellite  $n \in \mathcal{N}'$ , they observe the shard of RoI by rolling their cameras during their forward motion along the fixed orbit. Therefore, for two shards  $m$  and  $z$ , they can be sequentially observed by a satellite  $n \in \mathcal{N}'$  only if they satisfy the following constraint,

$$\text{C1: } \frac{|\theta_m^n - \sigma_z^n|}{\Delta t_{(m,z)}^n} \leq r_n^{\max}, \forall n \in \mathcal{N}', \quad (1)$$

where  $\theta_m^n$  represents the camera inclination of satellite  $n$  after observing shard  $m$ ,  $\sigma_z^n$  denotes the camera inclination required for satellite  $n$  to start observing shard  $z$ ,  $r_n^{\max}$  refers to the maximum rolling velocity and  $\Delta t_{(m,z)}^n$  is the time required for the satellite to orbit from shard  $m$  to  $z$ . To optimize the observable shard allocation, we define a directed acyclic graph (DAG)  $\mathcal{D}_n = \{\mathcal{V}_n, \mathcal{E}_n\}$  for each satellite  $n$  that represents a feasible observation sequence, factoring in satellite orbital and camera rotation properties [9], [19].  $\mathcal{V}_n$  denotes the observable shard set of satellite  $n$ , with two additional virtual vertices  $a_n$  and  $b_n$  denoting the common source and destination, respectively.  $\mathcal{E}_n$  denotes the edge set consisting of the observable edges connecting pairs of nodes in  $\mathcal{V}_n$ . Specifically, there exists an edge from  $a_n$  to each  $m \in \mathcal{V}_n$  and an edge from  $m \in \mathcal{V}_n$  to  $b_n$ . Moreover, based on constraint C1, the DAG  $\mathcal{D}_n$  can be established for each satellite  $n \in \mathcal{N}'$ , the weight of each edge  $(m, z) \in \mathcal{E}_n$  is denoted as  $w_{(m,z)} \in \{0, 1\}$ , where  $w_{(m,z)} = 1$  represent that the observation sequence from  $m$  to  $z$  is feasible, vice versa.

We denote by  $x_{\tau_l}^{nm}$  the time slot  $\tau_l$  at which satellite  $n$  observed shard  $m$ , where  $x_{\tau_l}^{nm} \in \{0, 1\}$ . For a more fine-grained consideration of shard observations, as the shard  $A$  in Fig. 2, according to the movement orbits of the satellite, only the image size of the shadow area needs to be taken if the ascending satellite is selected for observation, and the image size of the complete segment needs to be taken if the descending satellite is used for observation. Therefore, by properly selecting satellites to capture the shards, the size of the image can be effectively reduced, thus reducing the transmission and computation overhead. To this end, we denote the size of the shard  $m$  as  $H_m$  and the image size for the satellite  $n$  to capture the shard  $m$  as  $W_{nm}$ . Then we denote the content percentage for the satellite  $n$  to capture the shard  $m$  as  $R_{nm} = H_m/W_{nm}$ . Higher content percentage enables encapsulation of the entire shard within a smaller image size.

### B. Shard Splitting, Routing and Computation

Depending on the position of the shard  $m \in \mathcal{M}$  in the RoI and the assigned observation satellite  $n \in \mathcal{N}'$ , shard  $m$  will start to transmit and compute (e.g., DNN inference or MIC, etc) in LEO satellite networks at the corresponding time slot  $\tau_l$ . The final result will arrive at the designated ground station. To boost the processing efficiency of each shard, we split the observed shard  $m \in \mathcal{M}$  into  $K^{nm}$  subshards for more efficient transmission and computation over multiple paths. Let  $\pi_k^{nm}$  denote the  $k$ -th subshard of shard  $m$  captured by satellite  $n$  with size  $v_k^{nm}$ , where  $k \in \{1, \dots, K^{nm}\}$  and  $\sum_{k=1}^{K^{nm}} v_k^{nm} = W_{nm}$ .

As illustrated in Fig. 3(a), we model the computation of each subshard  $\pi_k^{nm}$  by a sequential task graph, which can represent such as the DNN inference or MIC process [5], [20]. The task

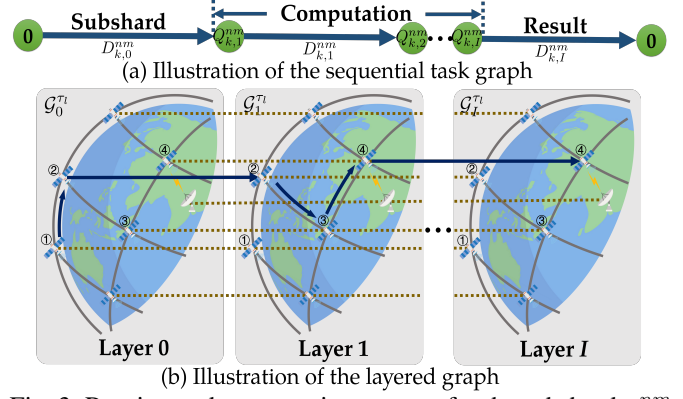


Fig. 3: Routing and computation process for the subshard  $\pi_k^{nm}$ .

graph is comprised of  $I$  layers, which correspond to the DNN model layers or image compression stages. Each vertex in the task graph represents a subtask, i.e., one layer of DNN or one stage of MIC. Moreover, to reflect the fact that the input data originated from the observation satellite and the result is routed to the destination ground station, two virtual subtasks 0 and  $I + 1$  are introduced to represent the entry and exit subtasks, respectively. Each edge in the task graph denotes that the input data of subtask  $i$  is the output of the preceding subtask  $i - 1$ . Let  $\rho_i$  denote the scaling factor, i.e., the scaling rate of the data size after computing the subtask  $i$ . Then the input data size of subtask  $i$  for the subshard  $\pi_k^{nm}$  can be represented as  $D_{k,i}^{nm} = \prod_{j=0}^{i-1} \rho_j v_k^{nm}$ , where  $i \in \{0, \dots, I\}$ . Besides, we denote the computation workload of subtask  $i$  for the subshard  $\pi_k^{nm}$  as  $Q_{k,i}^{nm} = C_i \prod_{j=0}^{i-1} \rho_j v_k^{nm}$ , where  $i \in \{1, \dots, I\}$  and  $C_i$  represents the CPU cycles required to compute 1-bit data of layer  $i$ . Note that  $C_i$  is dependent on the computation complexity of layer  $i$  [5]. Further, since subtasks 0 and  $I + 1$  represent the input and output of the DNN/MIC with  $I$  layers/stages, it can be obtained that  $Q_{k,0}^{nm} = Q_{k,I+1}^{nm} = 0$  and  $\rho_0 = 1$ .

To better represent the processing phase of each subshard  $\pi_k^{nm}$  and the queuing delay in LEO satellite networks, we construct a layered graph  $\mathcal{G}^{\tau_l} = (\mathcal{V}^{\tau_l}, \mathcal{E}^{\tau_l})$  to denote  $I + 1$  replications of the network topology  $\mathcal{G}_u^{\tau_l} = \{\mathcal{V}_u^{\tau_l}, \mathcal{E}_u^{\tau_l}\}$  at time slot  $\tau_l$  inspired by [21]. We will show in Subsection III-B that this modeling approach enables efficient routing and computation node selection while considering the queuing delay. As illustrated in Fig. 3(b), in layered graph  $\mathcal{G}^{\tau_l}$ , there are  $I + 1$  copies of  $\mathcal{G}_u^{\tau_l}$ , named  $\mathcal{G}_0^{\tau_l}, \dots, \mathcal{G}_I^{\tau_l}$  with  $\mathcal{G}_i^{\tau_l} = (\mathcal{V}_i^{\tau_l}, \mathcal{E}_i^{\tau_l}), \forall i \in \{0, \dots, I\}$ . Let  $e_i \in \mathcal{G}_i^{\tau_l}$  denote the replicated node of  $e \in \mathcal{G}_u^{\tau_l}$ , and hence,  $(e_i, f_i) \in \mathcal{G}_i^{\tau_l}$  denote the replicated link of  $(e, f) \in \mathcal{G}_u^{\tau_l}$ . Further, there is an edge from node  $e_{i-1}$  to  $e_i$  for all  $e \in \mathcal{G}^{\tau_l}$  and  $i \in \{1, \dots, I\}$ . These edges are denoted by  $\mathcal{E}_c^{\tau_l}$ . Accordingly, it can be obtained that  $\mathcal{V}^{\tau_l} = \mathcal{V}_0^{\tau_l} \cup \dots \cup \mathcal{V}_I^{\tau_l}$  and  $\mathcal{E}^{\tau_l} = \mathcal{E}_0^{\tau_l} \cup \dots \cup \mathcal{E}_I^{\tau_l} \cup \mathcal{E}_c^{\tau_l}$ .

We now discuss the routing and the computation node selection for each subshard  $\pi_k^{nm}$  in the layered graph  $\mathcal{G}^{\tau_l}$ . Depending on the observation scheduling strategy and the destination, we define the source and destination nodes for subshard  $\pi_k^{nm}$  are  $s_k^{nm} \in \mathcal{V}^{\tau_l}$  and  $t_k^{nm} \in \mathcal{V}^{\tau_l}$ , respectively. Then we need to find a path from the  $s_{k,0}^{nm}$  of layer 0 to the  $t_{k,I}^{nm}$  of layer  $I$  in the layered graph  $\mathcal{G}^{\tau_l}$ , where the cross-layer edge

represents the computation of the corresponding layer in the task graph. Specifically, if the routing traverses edge  $(e_{i-1}, e_i)$ , then layer  $i$  in the task graph is computed at node  $e$ . Moreover, if the routing traverses the edge  $(e_i, f_i)$ , then the output data of layer  $i$  in the task graph is transmitted from node  $e$  to node  $f$ . We denote the transmission capacity of edge  $(e, f) \in \mathcal{E}^{\tau_l}$  as  $\omega_{ef}$  and the computation capacity of satellite  $e$  as  $\mu_e$ .

To provide a clearer representation of the routing and computation process of subshard  $\pi_k^{nm}$  in the layered graph  $\mathcal{G}^{\tau_l}$ , we use Fig. 3 as an illustrative example. Note that Fig. 3(a) corresponds to Fig. 3(b), where the edges in Fig. 3(a) represent the layer  $i$  of the task transmission in the graph  $\mathcal{G}_i^{\tau_l}$ , and the vertexes in Fig. 3(a) represent the layer  $i$  of the task computation on the connecting edges of the graphs  $\mathcal{G}_{i-1}^{\tau_l}$  and  $\mathcal{G}_i^{\tau_l}$ . In Fig. 3(b), shard  $m$  is captured by satellite ①. After the shard splitting is completed, subshard  $\pi_k^{nm}$  will be routed and computed in layered graph  $\mathcal{G}^{\tau_l}$ . Specifically, the subshard  $\pi_k^{nm}$  is first transmitted from satellite ① to satellite ②. Then the computation for the 1-th layer of the task is performed on satellite ②. Subsequently, the computation results are transmitted to satellite ④ via satellite ③. Finally, satellite ④ completes the computation of the remaining layers and transmits the computation results to the ground station.

Referring to the priority setting in [22], we set the priority with the subshard as the smallest unit and assume preemption scheduling on the transmission links and computation nodes. Specifically, priority is assigned to every subshard and uniformly applied to all the layers in  $\mathcal{G}^{\tau_l}$  belonging to the same subshard. For instance, if subshards  $\pi_1^{nm}$  and  $\pi_2^{nm}$  utilize the same computation node or transmission link, with  $\pi_1^{nm}$  having higher priority, then  $\pi_2^{nm}$  must wait until  $\pi_1^{nm}$  completes its transmission or computation before it can proceed. Therefore, there is a queue for every transmission edge, and  $L_{ef}$  is the queue length at edge  $(e, f)$  representing the data size waiting to be transmitted. Further,  $L_e$  is the queue length of computation tasks waiting to be computed at node  $e$ . The above capacities and queue lengths in LEO satellite networks are reflected into the layered graph  $\mathcal{G}^{\tau_l}$  as follows: 1)  $L_{e_i, f_i} = L_{ef}$ ,  $\omega_{e_i, f_i} = \omega_{ef}$ ,  $\forall (e, f) \in \mathcal{E}_u^{\tau_l}$ ,  $i \in \{0, \dots, I\}$ ; 2)  $L_{e_{i-1}e_i} = L_e$ ,  $\mu_{e_{i-1}e_i} = \mu_e$ ,  $\forall e \in \mathcal{V}_u^{\tau_l}$ ,  $i \in \{1, \dots, I\}$ .

Let  $x_{k,ef}^{nm}$  denote the routing or computation node selection for each subtask of subshard  $\pi_k^{nm}$ , with a value of 1 if it traverses the edge  $(e, f) \in \mathcal{E}^{\tau_l}$  in  $\mathcal{G}^{\tau_l}$ , and 0 otherwise. Define  $x_{k,e}^{nm}$  as the indicator to represent whether the node  $e \in \mathcal{V}_u^{\tau_l}$  is selected to perform the computation for the subshard  $\pi_k^{nm}$ . Then the total time cost (i.e., transmission, computation and queuing delay) for the subshard  $\pi_k^{nm}$  can be denoted as

$$T_k^{nm} = \sum_{i=0}^I \sum_{(e,f) \in \mathcal{E}_i^{\tau_l}} \frac{\prod_{i=0}^i \rho_i v_k^{nm}}{\omega_{ef}} x_{k,ef}^{nm} + \sum_{(e,f) \in \mathcal{E}_c^{\tau_l}} \frac{C_i \prod_{i=0}^{i-1} \rho_i v_k^{nm}}{\mu_{ef}} x_{k,ef}^{nm} + \sum_{i=0}^I \sum_{(e,f) \in \mathcal{E}_i^{\tau_l}} \frac{L_{ef}}{\omega_{ef}} x_{k,ef}^{nm} + \sum_{e \in \mathcal{V}_u^{\tau_l}} \frac{L_e}{\mu_e} x_{k,e}^{nm}. \quad (2)$$

For the subshard  $\pi_k^{nm}$ , since a path from the source (i.e., observation satellite)  $s_k^{nm}$  to the destination (i.e., ground station)  $t_k^{nm}$  needs to be determined, together with computation

node selection, the following constraint need to be satisfied.

$$\mathbf{C2}: \sum_{f:(e,f) \in \mathcal{E}^{\tau_l}} x_{k,ef}^{nm} - \sum_{f:(f,e) \in \mathcal{E}^{\tau_l}} x_{k,fe}^{nm} = \begin{cases} 1, & \text{if } e = s_k^{nm}, \\ -1, & \text{if } e = t_k^{nm}, \\ 0, & \text{otherwise,} \end{cases} \quad (3)$$

where  $\forall k \in \{1, \dots, K^{nm}\}$ ,  $n \in \mathcal{N}'$ ,  $m \in \mathcal{M}$  and  $e \in \mathcal{V}^{\tau_l}$ .

### C. Problem Formulation

Then the observation scheduling, routing and computation node selection problem can be formulated as follows:

$$\begin{aligned} \min_{x_{k,ef}^{nm}, x_{k,e}^{nm}, x_{k,ef}^{nm}} \quad & \sum_{l=1}^T \max_{n \in \{1, \dots, V\}} \{x_{k, \tau_l}^{nm} \max_{k \in \{1, \dots, K^{nm}\}} T_k^{nm}\} \quad (\mathbf{P1}) \\ \text{s.t. } \quad & \mathbf{C1}, \mathbf{C2}, \mathbf{C3}: \sum_{k=1}^{K^{nm}} v_k^{nm} = W_{nm}, \forall n \in \mathcal{N}', m \in \mathcal{M}, \\ & \mathbf{C4}: v_k^{nm} \geq 0, \forall k \in \{1, \dots, K^{nm}\}, n \in \mathcal{N}', m \in \mathcal{M}, \\ & \mathbf{C5}: x_{k, \tau_l}^{nm} \in \{0, 1\}, \forall n \in \mathcal{N}', m \in \mathcal{M}, l \in \{1, \dots, T\}, \\ & \mathbf{C6}: x_{k,e}^{nm} \geq x_{k, e_{i-1}e_i}^{nm}, \forall k \in \{1, \dots, K^{nm}\}, \\ & \quad n \in \mathcal{N}', m \in \mathcal{M}, i \in \{1, \dots, I\}, e \in \mathcal{V}^{\tau_l}, \\ & \mathbf{C7}: x_{k,ef}^{nm} \in \{0, 1\}, \forall k \in \{1, \dots, K^{nm}\}, \\ & \quad n \in \mathcal{N}', m \in \mathcal{M}, (e, f) \in \mathcal{E}^{\tau_l}, \\ & \mathbf{C8}: x_{k,e}^{nm} \in \{0, 1\}, \forall k \in \{1, \dots, K^{nm}\}, \\ & \quad n \in \mathcal{N}', m \in \mathcal{M}, e \in \mathcal{V}^{\tau_l}. \end{aligned}$$

In the above, our objective is to minimize the time cost of routing and computing the observed shard in each time slot  $\tau_l$  to achieve real-time response (e.g., real-time object detection of each shard in a large RoI), where the inner max function represents the completion time of the subshard for the same shard, and the outer max function represents the completion time of the shard observed in the same time slot. Constraints **C3** and **C4** ensure that the size of the subshard after splitting is non-negative and sums to the size of the original shard. Constraints **C5**, **C6**, **C7** and **C8** ensure that  $x_{k,ef}^{nm}$  and  $x_{k,e}^{nm}$  are 0-1 integer variables and  $x_{k,e}^{nm}$  is 1 in the optimal solution only if node  $e$  is selected for the computation.

### III. SOLUTION OF THE PROBLEM

Solving problem **P1** faces several challenges. First of all, since problem **P1** is a mixed integer programming problem, it is NP-hard to be solved in general. Then the routing of different subshards would affect each other. Finally, since queuing delay is considered, it is difficult to solve directly for the shard splitting. To address the above challenges, we decouple the original problem into a two-stage optimization problem. The first stage is the optimization of multi-satellite observation scheduling. However, given the NP-hard nature of such combinatorial optimization problem, a centralized approach is difficult for optimization. Hence, we propose an efficient iterative algorithm relying on distributed inter-satellite interactions leveraging potential game theory [23]. The second stage is to optimize the processing of each shard. Specifically, for the smallest transmission unit (i.e., subshard) in LEO satellite networks, we first prove that the routing and computation node selection has a theoretically optimal solution under linear programming relaxation for the consideration of queuing delay under our modeling framework. In addition, we derive the theoretically optimal solution for shard splitting, and then we design an efficient iterative algorithm to jointly



determine the shard splitting, routing and computation node selection. Finally, we design a theoretically guaranteed system-wide greedy-based strategy to optimize the time cost for the case where multiple shards are captured in the same time slot, taking into account the queuing delay at the system level.

#### A. Solution for the Observation Scheduling

For shard observation scheduling, we first define the set of feasible observation policies for each satellite. Specifically, according to the DAG  $\mathcal{D}_n$  of each satellite  $n \in \mathcal{N}'$ , a feasible observation sequence from the start node  $a_n$  to the end node  $b_n$  forms a policy  $\mathbf{p}_n^f$ . All policies in  $\mathcal{D}_n$  comprise the policy set of satellite  $n$ , denoted by  $\mathcal{P}_n = \{\mathbf{p}_n^1, \mathbf{p}_n^2, \dots\}$ . Denote  $\mathbf{p}_n^a$  as the longest sequence in  $\mathcal{P}_n$  and  $\mathcal{O}(\mathbf{p}_n)$  as the set of observation shards on policy  $\mathbf{p}_n$ . Each satellite  $n \in \mathcal{N}'$  selects a policy  $\mathbf{p}_n \in \mathcal{P}_n$  to observe shards and these selected policies consist of the policy profile denoted by  $\mathbf{P} = (\mathbf{p}_1, \dots, \mathbf{p}_V)$ , where  $\mathbf{P} \in \mathcal{P}_1 \times \dots \times \mathcal{P}_V$ . Note that the value of  $x_{\tau_l}^{nm}$  can be obtained based on  $\mathbf{P}$ , i.e., it is available through  $\mathbf{P}$  at which time slot  $\tau_l$  which shard  $m$  is observed by the satellite  $n$ . Hence, we convert the decision variable for observation scheduling to  $\mathbf{P}$ . Define  $\sum_{l=1}^T \sum_{n=1}^V x_{\tau_l}^{nm} = |\{n | n \in \mathcal{N}', \mathbf{p}_n \in \mathbf{P}, m \in \mathcal{O}(\mathbf{p}_n)\}|$  as the number of satellites observing the same shard  $m$ .

The goal of observation scheduling is that multiple satellites work jointly to maximize the RoI coverage with the least repeated shards. To this end, the group of observation satellites needs to cover the maximum RoI while reducing the subsequent transmission and computation overhead. Accordingly, the reward for the satellite  $n$  to capture the shard  $m$  can be defined as the content percentage  $R_{nm}$  (i.e., effective RoI coverage in an observed image). Hence, we define the global utility function  $\Phi: \mathcal{P}_1 \times \dots \times \mathcal{P}_V \rightarrow \mathbb{R}$  as

$$\Phi(\mathbf{P}) = \sum_{n=1}^V \sum_{m \in \mathcal{O}(\mathbf{p}_n)} \frac{R_{nm}}{\sum_{l=1}^T \sum_{n=1}^V x_{\tau_l}^{nm}}, \quad (4)$$

where  $\sum_{m \in \mathcal{O}(\mathbf{p}_n)} \frac{R_{nm}}{\sum_{l=1}^T \sum_{n=1}^V x_{\tau_l}^{nm}}$  denotes that each observation satellite whose observation sequence  $\mathbf{p}_n$  include shard  $m$  contributes  $\frac{R_{nm}}{\sum_{l=1}^T \sum_{n=1}^V x_{\tau_l}^{nm}}$  to the global utility at  $m$ , which promotes more content percentages but discourage repeated shards in RoI coverage. Then the optimal observation sequence to maximize the global utility needs to be found, such that

$$\mathbf{P}^* = \arg \max_{\mathbf{P} \in \mathcal{P}_1 \times \dots \times \mathcal{P}_V} \Phi(\mathbf{P}). \quad (5)$$

To achieve the above goal, we solve the complicated observation scheduling problem by casting it as a strategic game  $G = (\mathcal{N}', \mathcal{P}_1 \times \dots \times \mathcal{P}_V, \{U_1, \dots, U_V\})$  and leverage potential game theory as the algorithm design tool, where  $\mathcal{P}_n$  and  $U_n$  are the action space and the utility for each satellite  $n \in \mathcal{N}'$ . The key point then lies in formulating the local utility function  $U_n$ , such that the satellites try to iteratively improve their local utilities to boost the global utility. This formulation ensures that the game  $G$  operates as an exact potential game, i.e., there is a guarantee of a Nash Equilibrium (NE), achievable by iterative and asynchronous better response update process (i.e., one player improves its local utility in each iteration) [23]. Before designing, we provide Definition 1 based on [23].

**Definition 1.** Let  $\mathbf{p}_{-n} = \{\mathbf{p}_1, \dots, \mathbf{p}_{n-1}, \mathbf{p}_{n+1}, \dots, \mathbf{p}_V\}$  denote the action profile of opponents for satellite  $n \in \mathcal{N}'$ .

#### Algorithm 1 Iterative Algorithm for Observation Scheduling

---

```

1: Initialization:  $T = 0$ ,  $A_n(0) = \mathbf{p}_n^a, \forall n \in \mathcal{N}'$  and  $A(0) = \{\mathbf{p}_n(0)\}_{n \in \mathcal{N}'}$ .
2: while  $A(T)$  is not NE do
3:   Select an observation satellite  $n \in \mathcal{N}'$  randomly.
4:   Satellite  $n$  selects an improved strategy  $A_n(T+1) = \mathbf{p}_n^i$ ,
     where  $\mathbf{p}_n^i \in \mathcal{P}_n$ .
5:   Update  $A_{-n}(T+1) = A_{-n}(T)$ .
6:    $A(T+1) = \{A_n(T+1), A_{-n}(T+1)\}$ .
7:    $T = T + 1$ .
8: end while
9: return  $\mathbf{P}^* = \{A_n(T)\}_{n \in \mathcal{N}'}$ .

```

---

Each satellite  $n \in \mathcal{N}'$  aims to select the best action  $\mathbf{p}_n^*$  to maximize its utility no matter what the  $\mathbf{p}_{-n}$  are. In game  $G$ , a solution reaches a NE if each satellite  $n \in \mathcal{N}'$  optimizes its local utility and no satellite can improve its utility by solely changing its actions. Further, game  $G$  is called an exact potential game if there exists a potential function  $\Phi: \{\mathcal{P}_n\}_{n \in \mathcal{N}'} \rightarrow \mathbb{R}$  such that,  $\mathbf{p}_n', \mathbf{p}_n'' \in \mathcal{P}_n, \mathbf{p}_{-n} \in \{\mathcal{P}_i\}_{i \neq n}, \forall n \in \mathcal{N}'$ :

$$U_n(\mathbf{p}_n', \mathbf{p}_{-n}) - U_n(\mathbf{p}_n'', \mathbf{p}_{-n}) = \Phi(\mathbf{p}_n', \mathbf{p}_{-n}) - \Phi(\mathbf{p}_n'', \mathbf{p}_{-n}). \quad (6)$$

In exact potential game  $G$ , NE (i.e.,  $\mathbf{P}^*$ ) is guaranteed to exist.

We then construct the local utility function for each satellite  $n \in \mathcal{N}'$  to form a potential game as follows.

**Theorem 1.** The game  $G$  is an exact potential game when the local utility function of each satellite  $n \in \mathcal{N}'$  is defined as

$$U_n(\mathbf{P}) = |\mathcal{O}(\mathbf{p}_n)| - \sum_{m \in \mathcal{O}(\mathbf{p}_n)} \left( \sum_{l=1}^T \sum_{n=1}^V x_{\tau_l}^{nm} - \frac{\sum_{n=1}^V R_{nm}}{\sum_{l=1}^T \sum_{n=1}^V x_{\tau_l}^{nm}} \right). \quad (7)$$

*Proof.* The key idea is to verify that the defined local utility of each satellite  $n \in \mathcal{N}'$  satisfies the condition in (6). The proof is given in Appendix A.  $\square$

We have that the local utility of satellite  $n \in \mathcal{N}'$  increases with the shard observation numbers  $|\mathcal{O}(\mathbf{p}_n)|$  and local reward  $R_{nm}$ , and decreases with the repeated observation numbers for the same shard according to Theorem 1. Based on the property of potential game [23], we then design an iterative algorithm for observation scheduling in Algorithm 1, such that in each iteration a satellite improves its observation sequence to boost its local utility, which can lead to the optimization of the global utility in (4) until the equilibrium point (e.g., no satellites can improve for a period of iterations) is reached. Note that the optimization process of observation scheduling only needs to be conducted once before the satellites carry out the actual EOM task operations until a new task is input.

#### B. Solution for the Shard Splitting, Routing and Computation Node Selection of Each Shard

1) *Solution for the routing and computation node selection:* We now perform the solution for the optimal routing and computation node selection of each subshard  $\pi_k^{nm}$ . We first assume that the splitting size of the shard is fixed and the subproblem is hence formulated as follows:

$$\min_{x_{k,ef}^{nm}, x_{k,e}^{nm}} T_k^{nm} \quad (\mathbf{P}_{\text{sub1}})$$

s.t. C2, C6 – C8.

Though the problem  $\mathbf{P}_{\text{sub1}}$  is NP-hard to be solved in general, we show in the following Theorem that it has 0-1 integer optimal solutions, i.e., the optimal solution can also be

found by relaxing constraints **C7** and **C8** to  $0 \leq x_{k,ef}^{nm} \leq 1$  and  $0 \leq x_{k,e}^{nm} \leq 1$ . According to the above, the problem  $\mathbf{P}_{\text{sub1}}$  can be solved in polynomial time.

**Theorem 2.** For the routing and computation node selection problem of subshard  $\pi_k^{nm}$ , it has integer optimal solutions after linear programming (LP) relaxation (i.e., **C7** and **C8** are relaxed to  $0 \leq x_{k,ef}^{nm} \leq 1$  and  $0 \leq x_{k,e}^{nm} \leq 1$ ).

*Proof.* The key idea is to prove that the formulation of  $\mathbf{P}_{\text{sub1}}$  has a special structure called **total unimodularity**, which ensures the integer optimal solutions after LP relaxation [24]. The proof is given in Appendix B.  $\square$

2) *Solution for the shard splitting:* To solve the shard splitting optimally, we first fix the routing and computation node selection policies for each subshard of shard  $m \in \mathcal{M}$  and define the set of the selected transmission links and computation nodes as  $\mathcal{X}_k^{nm}$ . Then without considering the queuing delay, the unit transmission and computation delay for the subshard  $\pi_k^{nm}$  can be denoted as

$$t_k^{nm} = \sum_{i=0}^I \sum_{(e,f) \in \mathcal{E}_i^{\tau_l} \cap \mathcal{X}_k^{nm}} \frac{\prod_{i=0}^I \rho_i}{\omega_{ef}} + \sum_{(e,f) \in \mathcal{E}_c^{\tau_l} \cap \mathcal{X}_k^{nm}} \frac{C_i \prod_{i=0}^{i-1} \rho_i}{\mu_{ef}}. \quad (8)$$

Due to the preemptive scheduling setting in Subsection II-B, the lower priority subshard needs to wait for the higher priority subshard during transmission and computation. So the lower priority subshard will incur queuing delay if it uses the same transmission link and computation node as in the higher priority subshard. Here we set the priority for the subshard as the index order. Later we will show that the initial setting of different priorities for subshards in a shard does not affect the final result. Let  $\mathcal{X}_{j,k}^{nm} = \mathcal{X}_j^{nm} \cap \mathcal{X}_k^{nm}$  denote the intersection of the policies with subshard  $\pi_j^{nm}$  and  $\pi_k^{nm}$ . Then for subshard  $\pi_k^{nm}$ , the unit queuing delay caused by the  $j$ -th subshard with higher priority than itself can be denoted as

$$t_{j,k}^{nm} = \sum_{i=0}^I \sum_{(e,f) \in \mathcal{E}_i^{\tau_l} \cap \mathcal{X}_{j,k}^{nm}} \frac{\prod_{i=0}^I \rho_i}{\omega_{ef}} + \sum_{(e,f) \in \mathcal{E}_c^{\tau_l} \cap \mathcal{X}_{j,k}^{nm}} \frac{C_i \prod_{i=0}^{i-1} \rho_i}{\mu_{ef}}, \quad (9)$$

where  $1 \leq j < k$ . Then we define a diagonal matrix  $\mathbf{D}^{nm}$  as  $\mathbf{D}^{nm} \triangleq \begin{pmatrix} t_1^{nm} & t_2^{nm} + \frac{v_1^{nm}}{v_2^{nm}} t_{1,2}^{nm} & \dots & t_{K^{nm}}^{nm} + \sum_{j=1}^{K^{nm}-1} \frac{v_j^{nm}}{v_{K^{nm}}^{nm}} t_{j,K^{nm}}^{nm} \end{pmatrix}$ . (10)

Since the priority is ordered by the index order of the subshard, each diagonal element in  $\mathbf{D}^{nm}$  represents the time cost of each subshard  $\pi_k^{nm}$  (i.e., transmission and computation delay plus queuing delay for occupying the same transmission link and computation node). Denote  $\mathbf{V}^{nm} \triangleq [v_1^{nm}, \dots, v_{K^{nm}}^{nm}]^T \in \mathbb{R}^{K^{nm}}$  as the variables regarding the splitting size of shard  $m$ . Then, the subproblem for shard splitting can be denoted as

$$\begin{aligned} \min_{\mathbf{V}^{nm}} \quad & \|\mathbf{D}^{nm} \mathbf{V}^{nm}\|_{\infty} \quad (\mathbf{P}_{\text{sub2}}) \\ \text{s.t.} \quad & \mathbf{D1} : \mathbf{1}^T \mathbf{V}^{nm} = W_{nm}, \\ & \mathbf{D2} : \mathbf{V}^{nm} \geq 0. \end{aligned}$$

$\mathbf{P}_{\text{sub2}}$  is an infinity norm minimization problem (i.e., the inner min-max function in  $\mathbf{P1}$ ), the slack form of which is feasible and its optimal objective value is finite. Therefore, the simplex method and interior point method can be applied to obtain the optimal solution efficiently. To reduce the time complexity, we derive the analytical expression of the optimal solution for  $\mathbf{P}_{\text{sub2}}$ . The result is as the following theorem.

**Algorithm 2** Iterative Algorithm for the Optimal Shard Splitting, Routing and Computation Node Selection for Each Shard

```

1: Initialization:  $E = 0$ ,  $L = 0$ ,  $\mathcal{X}_k^{nm}(0)$  and  $v_k^{nm}(0) = W_{nm}/K^{nm}$ ,  $\forall k \in \{1, \dots, K^{nm}\}$ .
2: while  $\mathcal{X}_k^{nm}(E-1) \neq \mathcal{X}_k^{nm}(E)$ ,  $\forall k \in \{1, \dots, K^{nm}\}$  do
3:    $E = E + 1$ .
4:   Solve problem  $\mathbf{P}_{\text{sub}}$  for subshard  $\pi_k^{nm}$ ,  $\forall k \in \{1, \dots, K^{nm}\}$ .
5:   Obtain policies  $\mathcal{X}_k^{nm}(E)$ ,  $\forall k \in \{1, \dots, K^{nm}\}$  from line 4.
6:   Update  $\mathbf{D}^{nm}(E)$  according to the line 5.
7:   while  $\mathbf{V}^{nm}(L-1) \neq \mathbf{V}^{nm}(L)$  do
8:      $L = L + 1$ .
9:     Calculate the value in equation (12).
10:    Update  $v_k^{nm}(L)$ ,  $\forall k \in \{1, \dots, K^{nm}\}$  based on equation (13).
11:   end while
12: end while
13: return  $v_k^{nm*} = v_k^{nm}(L)$  and  $\mathcal{X}_k^{nm*} = \mathcal{X}_k^{nm}(E)$ ,  $\forall k \in \{1, \dots, K^{nm}\}$ .

```

**Theorem 3.** The optimal objective value of  $\mathbf{P}_{\text{sub2}}$  is

$$\min_{\mathbf{V}^{nm}} \|\mathbf{D}^{nm} \mathbf{V}^{nm}\|_{\infty} = \frac{W_{nm}}{\sum_{k=1}^{K^{nm}} 1/D_{k,k}^{nm}}, \quad (11)$$

if

$$D_{i,i}^{nm} v_i^{nm} = D_{j,j}^{nm} v_j^{nm}, 1 \leq i \neq j \leq K^{nm}, \quad (12)$$

where  $v_i^{nm}$  is the  $i$ -th element of vector  $\mathbf{V}^{nm}$  and  $D_{i,i}^{nm}$  is the  $i$ -th diagonal element of  $\mathbf{D}^{nm}$ .

*Proof.* For  $\mathbf{P}_{\text{sub2}}$ , since  $D_{k,k}^{nm} > 0$ ,  $v_k^{nm} \geq 0$ ,  $\forall k$ , we have

$$\|\mathbf{D}^{nm} \mathbf{V}^{nm}\|_{\infty} \triangleq \max_k \{D_{k,k}^{nm} v_k^{nm}\} = \lim_{x \rightarrow \infty} \sqrt[x]{\sum_{k=1}^{K^{nm}} (D_{k,k}^{nm} v_k^{nm})^x}, \quad (13)$$

According to the AM-GM inequality [25], the following inequality always holds:

$$\frac{\sum_{k=1}^{K^{nm}} (D_{k,k}^{nm} v_k^{nm})^x}{K^{nm}} \geq K^{nm} \sqrt[K^{nm}]{\prod_{k=1}^{K^{nm}} (D_{k,k}^{nm} v_k^{nm})^x}, \quad (14)$$

if (12) is satisfied, for  $\forall x > 0$ , we can get that

$$\sqrt[x]{\frac{\sum_{k=1}^{K^{nm}} (D_{k,k}^{nm} v_k^{nm})^x}{K^{nm}}} \geq K^{nm} \sqrt[K^{nm}]{\prod_{k=1}^{K^{nm}} D_{k,k}^{nm} v_k^{nm}}. \quad (15)$$

Multiply both sides by  $\sqrt[x]{K^{nm}}$  and take the limit, we have

$$\|\mathbf{D}^{nm} \mathbf{V}^{nm}\|_{\infty} \geq \lim_{x \rightarrow \infty} \sqrt[x]{K^{nm}} \cdot K^{nm} \sqrt[K^{nm}]{\prod_{k=1}^{K^{nm}} D_{k,k}^{nm} v_k^{nm}}. \quad (16)$$

Combining with  $\mathbf{P}_{\text{sub2}}$  and (12), the right side of (16) is actually a constant. Then we can obtain that

$$\begin{aligned} \min_{\mathbf{V}^{nm}} \|\mathbf{D}^{nm} \mathbf{V}^{nm}\|_{\infty} &= \lim_{x \rightarrow \infty} \sqrt[x]{K^{nm}} \cdot K^{nm} \sqrt[K^{nm}]{\prod_{k=1}^{K^{nm}} D_{k,k}^{nm} v_k^{nm}} \\ &= K^{nm} \sqrt[K^{nm}]{\prod_{k=1}^{K^{nm}} D_{k,k}^{nm} v_k^{nm}} = \frac{W_{nm}}{\sum_{k=1}^{K^{nm}} 1/D_{k,k}^{nm}}. \quad (17) \end{aligned}$$

It can be obtained from the above that (11) and (12) are the optimal objective value and corresponding optimal condition of  $\mathbf{P}_{\text{sub2}}$ , respectively. This completes the proof.  $\square$

According to Theorem 3, since the final optimal solution is that each subshard has the same time cost considering the queuing delay, it will eventually converge to the same time cost regardless of the initial priority setting of different subshards. Based on Theorems 2-3, we design the iterative Algorithm 2 to solve the size of the optimal shard splitting and obtain the solution for the optimal routing and computation node selection policies for each subshard  $\pi_k^{nm}$ .

### C. System-wide Greedy-based Routing and Computation Node Selection Policies

In this section, we focus on the case of multiple shard arrivals in the same time slot  $\tau_l$ . To this end, we develop a corresponding greedy-based scheme for minimizing the time cost while considering the queuing delay at the system level. We denote  $\mathcal{L}^{\tau_l} = \{L_{ef}^{\tau_l}, \forall (e, f) \in \mathcal{E}^{\tau_l}\}$  as the set of queuing tasks in LEO satellite networks at time slot  $\tau_l$ . Let  $s \in \mathcal{S}^{\tau_l}$  denote the shard captured in the time slot  $\tau_l$ , where  $\mathcal{S}^{\tau_l} = \{1, \dots, S^{\tau_l}\}$  and  $S^{\tau_l} = \sum_{n=1}^V \sum_{m=1}^M x_{\tau_l}^{nm}$ . Note that the initialization of set  $\mathcal{S}^{\tau_l}, \forall l \in \{1, \dots, T\}$  can be obtained according to the Algorithm 1. As illustrated in Algorithm 3, based on the dynamic property of LEO satellite networks, we solve for routing and computation node selection for shards arriving at each time slot  $\tau_l$  in time sequence. Note that if only one shard is captured in a time slot, the shard splitting number  $K^{nm}$  can be determined through a preset experiment. Specifically, as will be shown in Experiment (5) of Section IV, a large shard splitting number leads to dominating network queuing delay and increased time cost. Therefore, the optimal shard splitting number for each shard can be determined via preset experiments. Further, the routing and computation node selection policy can be directly obtained from Algorithm 2.

If more than one shard is captured in the same time slot, we propose a greedy-based scheme to determine the priority for each shard. Denote  $s_j$  as the  $j$ -th shard that performs the routing and computation node selection policies under the greedy scheme, where  $j \in \{1, \dots, J\}$  and  $J = S^{\tau_l}$ . Additionally, let  $K^{s_j}$  represent the splitting number for the  $j$ -th shard, the optimal value of which can be obtained through a preset experiment. Further, we define  $U_{K^{s_j}}^{\tau_l}(\mathcal{L}^{\tau_l})$  as the time cost for shard  $s_j$  arrived in the time slot  $\tau_l$  when it is routed on the basis of the formulation in the presence of queuing tasks  $\mathcal{L}^{\tau_l}$  under the shard splitting number is  $K^{s_j}$ . Then, in each round of our proposed algorithm, the shard with the smallest time cost is selected to give the highest priority in time slot  $\tau_l$ . After that, the set of queuing tasks is updated, and the routing and computation node selection for the remaining shards in the same time slot is based on the updated  $\mathcal{L}^{\tau_l}$ . The above procedure is repeated until all the shards in the same time slot have completed solving the routing and computation node selection policies. Notably, the ground station can function as the coordinator to perform the Algorithm 3 and broadcast the results to LEO satellites for distributed execution.

We further analyze the performance (i.e., approximation ratio) of Algorithm 3 regarding the optimal solution as follows.

**Theorem 4.** We denote  $Z_s^H$  and  $Z_s^L$  as the longest and shortest path lengths in hop count between the source node and destination node, respectively. Further, let  $Z^H = \max_s Z_s^H$  and  $Z^L = \max_s Z_s^L$ . Suppose each satellite in LEO satellite networks has  $A$  antennas (i.e.,  $\mathcal{G}_u^{\tau_l}$  is  $A$ -edge-connected). The time cost under the greedy algorithm in Algorithm 3 is at most  $\alpha T^{\tau_l*}$ , where  $T^{\tau_l*}$  is the optimal time cost in time slot  $\tau_l$  and

$$\alpha = \left(2 - \frac{1}{|\mathcal{V}_u^{\tau_l}| + |\mathcal{E}_u^{\tau_l}|}\right) \max \left\{ \left(1 + \frac{|\mathcal{E}_u^{\tau_l}|}{|\mathcal{V}_u^{\tau_l}|}\right) \frac{\max_e \mu_e}{\min_e \mu_e}, \right.$$

### Algorithm 3 System-wide Greedy-based Routing and Computation Node Selection Algorithm

```

1: Input: Observation scheduling strategy  $x_{\tau_l}^{nm}$ , where  $n \in \mathcal{N}'$ ,  $m \in \mathcal{M}$  and  $l \in \{1, \dots, T\}$ .
2: Initialization:  $j = 0$ ,  $L_{ef}^{\tau_l} = 0$ ,  $\forall (e, f) \in \mathcal{E}^{\tau_l}$ ,  $\mathcal{S}^{\tau_l}$ , where  $l \in \{1, \dots, T\}$ .
3: function QUEUING DELAY COMPUTING ( )
4:   Obtain  $v_k^{nm}, \mathcal{X}_k^{nm}, \forall k \in \{1, \dots, K^{nm}\}$  from Algorithm 2.
5:   for  $k = 1, \dots, K^{nm}$  do
6:      $L_{ef}^{\tau_l} \leftarrow L_{ef}^{\tau_l} + x_{k,ef}^{nm} D_{k,i}^{nm}, \forall (e, f) \in \mathcal{E}^{\tau_l}, i \in \{0, \dots, I\}$ .
7:      $L_e^{\tau_l} \leftarrow L_e^{\tau_l} + x_{k,ef}^{nm} Q_{k,i}^{nm}, \forall (e, f) \in \mathcal{E}_e^{\tau_l}, i \in \{1, \dots, I\}$ .
8:   end for
9: end function
10: for  $l = 1, \dots, T$  do
11:   if  $S^{\tau_l} = 0$  then continue.
12:   else if  $S^{\tau_l} = 1$  then
13:     Obtain  $K^{nm}$  through the preset experiment.
14:     Obtain  $v_k^{nm}, \mathcal{X}_k^{nm}, \forall k \in \{1, \dots, K^{nm}\}$  from Algorithm 2.
15:   else
16:      $j = 0$ .
17:     while  $S^{\tau_l} \neq \emptyset$  do
18:        $j = j + 1$ .
19:       Obtain  $K^{s_j}$  by the preset experiment,  $\forall s \in \mathcal{S}^{\tau_l}$ .
20:        $s_j = \arg \min_{s \in \mathcal{S}^{\tau_l}} U_{K^{s_j}}^{\tau_l}(\mathcal{L}^{\tau_l})$ .
21:        $\mathcal{L}^{\tau_l} \leftarrow \text{QUEUING DELAY COMPUTING}()$ .
22:        $S^{\tau_l} \leftarrow S^{\tau_l} \setminus \{s_j\}$ .
23:     end while
24:   end if
25: end for
26: return  $v_k^{nm}$  and  $\mathcal{X}_k^{nm}, \forall k \in \{1, \dots, K^{nm}\}, n \in \mathcal{N}', m \in \mathcal{M}$ .

```

$$\left(2, \frac{2(I+1)(|\mathcal{V}_u^{\tau_l}| + |\mathcal{E}_u^{\tau_l}|)}{A}\right) \frac{Z^H \max_{s,k,i} v_k^s \max_{(e,f)} \omega_{ef}}{Z^L \min_{s,k,i} v_k^s \min_{(e,f)} \omega_{ef}} \}. \quad (18)$$

*Proof.* We introduce the Minimum Service Time (MST) policy, which achieves the lowest transmission and computation time for each shard  $s$  with the time cost  $F_s^{MST}$ , and we can obtain that  $\frac{1}{|\mathcal{V}_u^{\tau_l}| + |\mathcal{E}_u^{\tau_l}|} \sum_{s=1}^{S^{\tau_l}} F_s^{MST} \leq T^{\tau_l*}$ . Moreover, we introduce the Minimum Queuing Time (MQT) policy, which means that  $J-1$  shards have been optimized in  $\tau_l$  by Algorithm 3, the  $J$ -th shard is assigned to a node  $e^*$  with the minimum computation queuing time. Note that the time cost of Algorithm 3 will not exceed MQT. Define  $\alpha_{tran} = \frac{Z^H \max_{s,k,i} v_k^s \max_{(e,f)} \omega_{ef}}{Z^L \min_{s,k,i} v_k^s \min_{(e,f)} \omega_{ef}}$  and  $\alpha_{comp} = \frac{\max_e \mu_e}{\min_e \mu_e}$ . We can get the upper bound for the transmission time plus computation time in MQT is  $\max(2\alpha_{tran}, \alpha_{comp}) T^{\tau_l*}$  and the upper bound for the queuing time in MQT is  $\max\left(\frac{2(I+1)\alpha_{tran}}{A}, \frac{\alpha_{comp}}{|\mathcal{V}_u^{\tau_l}|}\right) \sum_{j=1}^{J-1} F_{s_j}^{MST}$ . Combining the above, we can prove Theorem 4. The detailed proof is given in Appendix C.  $\square$

### IV. PERFORMANCE EVALUATION

In this section, we will present experiment results based on real-world datasets to verify the performance enhancement of our proposed framework. Specifically, we consider a Starlink constellation of 200 satellites evenly distributed over 20 orbits by observing Hong Kong and routing the observed shards and computing results to the ground station in Shanghai. The satellite dynamic characteristics are captured by Satellite Tool Kit (STK) simulation software. For the observation, as shown in Fig. 4, we assigned 10 satellites to observe Hong Kong, the green ones are ascending satellites and the orange ones are

TABLE I: Simulation Parameters.

| Parameter                                 | Value         |
|-------------------------------------------|---------------|
| Number of LEO satellites                  | 200           |
| Number of observation satellites          | 10            |
| Number of antennas for each LEO satellite | 4             |
| Inter-satellite communication rate        | [1, 1.5] Gbps |
| Satellite-to-ground communication rate    | [0.8, 1] Gbps |
| Satellite computation capability          | [2.5, 6] MHz  |
| Ground station computation capability     | [8, 20] MHz   |
| Image resolution for captured shard       | 200 Mb        |

descending satellites. The observation area of these satellites cut Hong Kong into 21 shards and the observation area of the satellite can be switched between three adjacent strips. According to the settings in [26], the important simulation parameters are listed in Table I. In addition, we consider two computation tasks, which are described as follows.

- **Multi-stage image compression (MIC) [27]:** We consider a five-stage image compression task, where the compression rate for each stage is 0.8, 0.7, 0.5, 0.3 and 0.1 of the original captured image, respectively.
- **Object detection by YOLO (OD) [20]:** We utilize YOLO as the detection network, which has 24 convolutional layers followed by 2 fully connected layers. The detailed network parameters can be found in [20].

To evaluate the performance of the proposed framework, the following baselines are compared.

- **NoSplit:** Instead of splitting the shard, this scheme directly transmits and computes the captured shard.
- **AvgSplit [16]:** This scheme evenly splits the captured shard for transmission and computation (*i.e.*, the size of each subshard is identical for a shard.).
- **NoQueue [28]:** This scheme performs our framework without considering queuing delay optimization.
- **Longest [5]:** In this scheme, each satellite captures its own entire strip without switching strips.

1) *Observation Scheduling:* Fig. 5 and Fig. 6 illustrate the coverage and observation size for different schemes under different amounts of observation satellites. To validate the scalability of our proposed framework, we set up two observation cases. The first one is that the satellite can continuously capture shards, *i.e.*, the rolling velocity of the satellite can be infinite, called case 1. The other one is that if the satellite needs to switch the stripe after capturing the current shard, the maximum rolling velocity of the satellite is limited to  $\frac{\pi}{6} \text{ rad/s}$ , called case 2. In addition, for the scenario with 6 satellites observing, the satellites with the serial numbers 0, 2, 4, 5, 7 and 9 in Fig. 4 are assigned to observe. It should be noted that given the limited number of satellites, no observation scheduling can achieve complete coverage of the RoI. It can be observed from Fig. 5 and Fig. 6 that SECO reduces the number of repeated shards and thus the total captured image size with improved coverage compared to Longest. Moreover, SECO shows superior performance for any observation cases and amounts of observation satellites, demonstrating its robustness.

2) *Impact of image resolution on the time cost:* In Fig. 7



Fig. 4: Illustration of observing Hong Kong by Starlink.

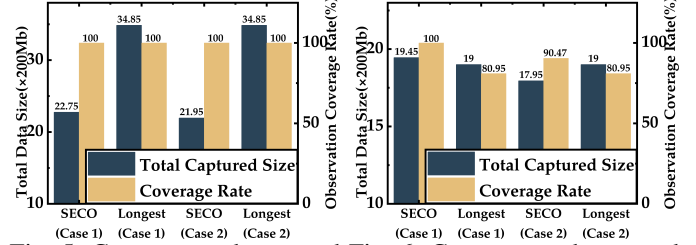


Fig. 5: Coverage and captured size (10 observation satellites).

and Fig. 8, we can observe that the time cost increases with higher image resolutions. This is primarily due to the increased load on both transmission and computation when dealing with larger image sizes. Moreover, the absence of shard splitting results in the highest time cost, clearly highlighting the indispensability of this technique. Furthermore, we can observe that the adoption of evenly splitting is non-optimal when considering queuing delay, which proves the effectiveness of the comprehensive optimization strategy adopted by SECO. Additionally, SECO surpasses all baseline methods in both MIC and OD tasks, demonstrating its superior performance.

3) *Impact of computation capacity on the time cost:* It can be observed in Fig. 9 and Fig. 10 that the time cost exhibits a clear inverse relationship with satellite computation capacity, indicating that an increase in computation capacity leads to a reduction in time cost for both the MIC and OD tasks. This can be attributed to the fact that higher computation capacity enhances system performance, resulting in decreased time costs. Moreover, the figures provide further evidence that the time cost without queuing delay optimization is significantly higher compared to our framework, thus underscoring the imperative nature of optimizing queuing delay.

4) *Impact of communication capacity on the time cost:* From Fig. 11 and Fig. 12, it can be observed that there is a reduction in time cost as the communication capacity increases. This phenomenon can be attributed to the enhanced hardware capacity, which improves the efficiency of both queuing and service, consequently reducing the overall time cost. Notably, the optimization effect of our framework is more pronounced in the OD task compared to the baseline, thereby validating the robust optimization performance of our framework in complex computation tasks.

5) *Impact of the shard splitting number on time cost:* We can observe from Fig. 13 that the time cost shows a decreasing



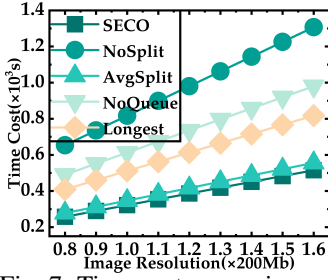


Fig. 7: Time cost versus image resolutions (MIC).

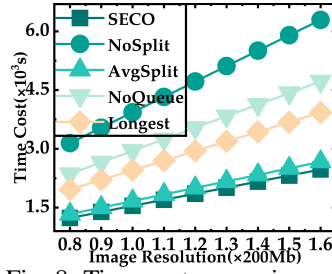


Fig. 8: Time cost versus image resolutions (OD).

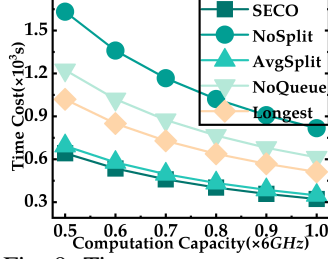


Fig. 9: Time cost versus computation capacities (MIC).

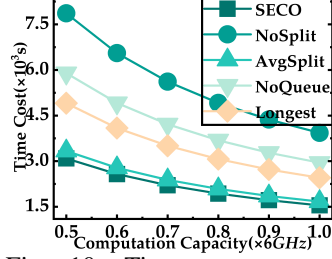


Fig. 10: Time cost versus computation capacities (OD).

trend initially, followed by an increase as the shard splitting number increases, both for the MIC and OD tasks. This can be attributed to the occurrence of significant queuing phenomena in the LEO satellite network when the shard splitting number becomes excessively large, leading to a dominant queuing delay. Therefore, a preliminary experiment can be conducted to determine the optimal shard splitting number.

6) *Impact of the number of observation satellites on time cost*: According to Fig. 14, our framework exhibits superior performance compared to all baseline models in scenarios involving different numbers of observation satellites. Remarkably, our framework achieves a substantial reduction in time cost, up to 60.7%, while ensuring 100% observation coverage. Moreover, our framework delivers significant optimization outcomes for both the MIC and OD tasks, thereby validating the effectiveness of observation scheduling, shard splitting and optimization strategies that consider queuing delay.

## V. RELATED WORKS

Recent advancements in free-space optical (FSO) communications technology [29] have enabled the establishment of ISLs within LEO satellite networks. Consequently, complete SEC services can be realized within the LEO satellite networks. Several research efforts have explored the potential of SEC technology, particularly in addressing computation challenges for both terrestrial and space missions [2], [26], [30]–[32].

Current Earth observation mission (EOM) advancements facilitate various tasks such as disaster monitoring and resource exploration [8]. Traditional observation scheduling approaches typically focus on capturing multiple RoIs, with each RoI assigned to a single satellite for imaging [6], [9], [10]. Lv *et al.* [6] presented a holistic solution that precisely models the observation service of a single LEO satellite and allocates several Earth observation tasks to a group of LEO satellites. An iterative algorithm is proposed in [9] to coordinate satellite observation resources to schedule observations for multiple

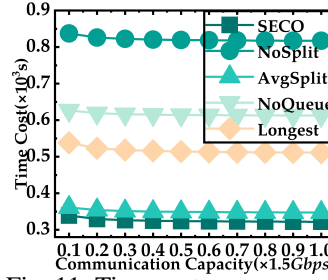


Fig. 11: Time cost versus communication capacities (MIC).

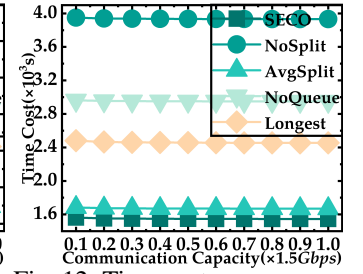


Fig. 12: Time cost versus communication capacities (OD).

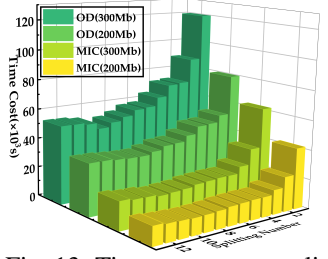


Fig. 13: Time cost versus split-numbers for a shard.

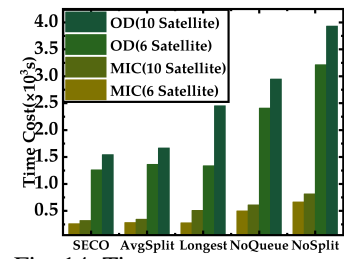


Fig. 14: Time cost versus number of observation satellites.

RoIs. However, the above efforts do not allow for the scheduling of fine-grained observations for a large RoI area. In [5], a distributed compression scheme is designed for a large RoI area with high resolution. However, this scheme does not consider the scheduling of observations within the RoI, and each satellite captures a complete shot based on its trajectory.

Various studies have aimed to tackle routing and computation node selection in resource-limited edges for complex inter-task dependent functions. Liu *et al.* [33] proposed a framework that forms such functions as a service function chain, and the computation node selection for each function is obtained by minimizing the total time cost. In [17], a dependent function embedding algorithm has been designed that considers both routing and computation node selection. However, the congestion in the network is not considered in the above work. Jung *et al.* [22] proposed a framework that optimizes the routing for DNN inference jobs in distributed networks. Nonetheless, there is still a need to incorporate congestion considerations and explore the potential benefits of raw image splitting for parallel transmission and computation.

## VI. CONCLUSION

In this paper, we propose a SEC-enabled framework that jointly optimizes observation scheduling, routing and computation node selection for wide-area and real-time EOM (SECO). SECO incorporates a distributed observation scheduling strategy based on game theory. This approach enables fine-grained boundary-side scheduling and multi-satellite cooperative observations of a large RoI area. To enhance efficiency, shard splitting is introduced to facilitate parallel transmission and computation. Regarding routing and computation node selection, SECO considers queuing delay within the LEO satellite networks during the optimization process. Further, we propose a theoretically guaranteed system-wide greedy-based strategy to minimize the time cost while considering queuing delay at a system level for simultaneous shard capturing.

## REFERENCES

- [1] Renchao Xie, Qinqin Tang, Qiuning Wang, Xu Liu, F. Richard Yu, and Tao Huang. Satellite-terrestrial integrated edge computing networks: Architecture, challenges, and open issues. *IEEE Network*, 34(3):224–231, 2020.
- [2] Xinyuan Zhang, Jiang Liu, Ran Zhang, Yudong Huang, Jincheng Tong, Ning Xin, Liang Liu, and Zehui Xiong. Energy-efficient computation peer offloading in satellite edge computing networks. *IEEE Transactions on Mobile Computing*, pages 1–15, 2023.
- [3] Jehyun Heo, Seungwoo Sung, Hyunwoo Lee, Incheol Hwang, and Daesik Hong. Mimo satellite communication systems: A survey from the phy layer perspective. *IEEE Communications Surveys & Tutorials*, 2023.
- [4] M. Harris. Tech giants race to build orbital internet [news]. *IEEE Spectrum*, 55(6):10–11, 2018.
- [5] Israel Leyva-Mayorga, Marc Martinez-Gost, Marco Moretti, Ana Pe  ez-Neira, Miguel Angel V  zquez, Petar Popovski, and Beatriz Soret. Satellite edge computing for real-time and very-high resolution earth observation. *IEEE Transactions on Communications*, 2023.
- [6] Mingsong Lv, Xuemei Peng, Wenjing Xie, and Nan Guan. Task allocation for real-time earth observation service with leo satellites. In *2022 IEEE Real-Time Systems Symposium (RTSS)*, pages 14–26. IEEE, 2022.
- [7] Camile Sothe, Alemu Gonsamo, Joyce Arabian, and James Snider. Large scale mapping of soil organic carbon concentration with 3d machine learning and satellite observations. *Geoderma*, 405:115402, 2022.
- [8] Ivan Franch-Pardo, Brian M Napoletano, Fernando Rosete-Verges, and Lawal Billa. Spatial analysis and gis in the study of covid-19. a review. *Science of the total environment*, 739:140033, 2020.
- [9] Yu Wang, Min Sheng, Weihua Zhuang, Shan Zhang, Ning Zhang, Runzi Liu, and Jiandong Li. Multi-resource coordinate scheduling for earth observation in space information networks. *IEEE Journal on Selected Areas in Communications*, 36(2):268–279, 2018.
- [10] Wei Liu, Huiting Yang, and Jiandong Li. Multi-functional time expanded graph: A unified graph model for communication, storage and computation for dynamic networks over time. *IEEE Journal on Selected Areas in Communications*, 2023.
- [11] Qiang Zhao, Le Yu, Zhenrong Du, Dailiang Peng, Pengyu Hao, Yongguang Zhang, and Peng Gong. An overview of the applications of earth observation satellite data: impacts and future trends. *Remote Sensing*, 14(8):1863, 2022.
- [12] Zhishu Shen, Jiong Jin, Cheng Tan, Atsushi Tagami, Shangguang Wang, Qing Li, Qiushi Zheng, and Jingling Yuan. A survey of next-generation computing technologies in space-air-ground integrated networks. *ACM Computing Surveys*, 2023.
- [13] Van-Phuc Bui, Thanh Q Dinh, Israel Leyva-Mayorga, Shashi Raj Pandey, Eva Lagunas, and Petar Popovski. On-board change detection for resource-efficient earth observation with leo satellites. *arXiv preprint arXiv:2305.10119*, 2023.
- [14] Bing Zhang, Yuanfeng Wu, Boya Zhao, Jocelyn Chanussot, Danfeng Hong, Jing Yao, and Lianru Gao. Progress and challenges in intelligent remote sensing satellite systems. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 15:1814–1822, 2022.
- [15] Roberto Cordone, Federico Gandellini, and Giovanni Righini. Solving the swath segment selection problem through lagrangean relaxation. *Computers & operations research*, 35(3):854–862, 2008.
- [16] Wuyang Zhang, Zhezhi He, Luyang Liu, Zhenhua Jia, Yunxin Liu, Marco Gruteser, Dipankar Raychaudhuri, and Yanyong Zhang. Elf: accelerate high-resolution mobile deep vision with content-aware parallel offloading. In *Proceedings of the 27th Annual International Conference on Mobile Computing and Networking*, pages 201–214, 2021.
- [17] Shuiguang Deng, Hailiang Zhao, Zhengzhe Xiang, Cheng Zhang, Rong Jiang, Ying Li, Jianwei Yin, Shahram Dustdar, and Albert Y Zomaya. Dependent function embedding for distributed serverless edge computing. *IEEE Transactions on Parallel and Distributed Systems*, 33(10):2346–2357, 2021.
- [18] Mario Minardi, Youssouf Drif, Thang X Vu, Ilora Maity, Christos Politis, and Symeon Chatzinotas. Sdn-based testbed for emerging use cases in beyond 5g ntn-terrestrial networks. In *NOMS 2023-2023 IEEE/IFIP Network Operations and Management Symposium*, pages 1–6. IEEE, 2023.
- [19] Virginie Gabrel, Alain Moulet, C  cile Murat, and Vangelis Th Paschos. A new single model and derived algorithms for the satellite shot planning problem using graph theory concepts. *Annals of Operations Research*, 69(0):115–134, 1997.
- [20] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 779–788, 2016.
- [21] Jianan Zhang, Abhishek Sinha, Jaime Llorca, Antonia M Tulino, and Eytan Modiano. Optimal control of distributed computing networks with mixed-cast traffic flows. *IEEE/ACM Transactions on Networking*, 29(4):1760–1773, 2021.
- [22] Sehun Jung and Hyang-Won Lee. Optimization framework for splitting dnn inference jobs over computing networks. *Computer Networks*, page 109814, 2023.
- [23] Dov Monderer and Lloyd S Shapley. Potential games. *Games and economic behavior*, 14(1):124–143, 1996.
- [24] Laurence A Wolsey and George L Nemhauser. *Integer and combinatorial optimization*, volume 55. John Wiley & Sons, 1999.
- [25] Zdravko Cvetkovski. *Inequalities: theorems, techniques and selected problems*. Springer Science & Business Media, 2012.
- [26] Qingze Fang, Zhiwei Zhai, Shuai Yu, Qiong Wu, Xiaowen Gong, and Xu Chen. Olive branch learning: A topology-aware federated learning framework for space-air-ground integrated network. *IEEE Transactions on Wireless Communications*, 2022.
- [27] Xiaoyang Li, Changsheng You, Sergey Andreev, Yi Gong, and Kaibin Huang. Wirelessly powered crowd sensing: Joint power transfer, sensing, compression, and transmission. *IEEE Journal on Selected Areas in Communications*, 37(2):391–406, 2018.
- [28] Jiaqi Cao, Shengli Zhang, Qingxia Chen, Houtian Wang, Mingzhe Wang, and Naijin Liu. Computing-aware routing for leo satellite networks: A transmission and computation integration approach. *arXiv preprint arXiv:2211.08820*, 2022.
- [29] Peng Hu. A cross-layer descent approach for resilient network operations of proliferated leo satellites. In *2023 IEEE Wireless Communications and Networking Conference (WCNC)*, pages 1–6. IEEE, 2023.
- [30] Lei Cheng, Gang Feng, Yao Sun, Mengjie Liu, and Shuang Qin. Dynamic computation offloading in satellite edge computing. In *ICC 2022-IEEE International Conference on Communications*, pages 4721–4726. IEEE, 2022.
- [31] Francesco Valente, Francesco G Lavacca, and Vincenzo Eramo. A resource allocation strategy in earth observation orbital edge computing-enabled satellite networks to minimize ground station energy consumption. In *NOMS 2023-2023 IEEE/IFIP Network Operations and Management Symposium*, pages 1–6. IEEE, 2023.
- [32] Xuelin Cao, Bo Yang, Yulong Shen, Chau Yuen, Yan Zhang, Zhu Han, H Vincent Poor, and Lajos Hanzo. Edge-assisted multi-layer offloading optimization of leo satellite-terrestrial integrated networks. *IEEE Journal on Selected Areas in Communications*, 41(2):381–398, 2022.
- [33] Liuyan Liu, Haisheng Tan, Shaofeng H-C Jiang, Zhenhua Han, Xiang-Yang Li, and Hong Huang. Dependent task placement and scheduling with function configuration in edge computing. In *Proceedings of the International Symposium on Quality of Service*, pages 1–10, 2019.

APPENDIX A  
PROOF OF THEOREM 1

To prove Theorem 1, we first define the potential function  $\Phi : \mathcal{P}_1 \times \dots \times \mathcal{P}_V \rightarrow \mathbb{R}$  as follows,

$$\begin{aligned} \Phi(\mathbf{P}) &= \sum_{n=1}^V \sum_{m \in \mathcal{O}(\mathbf{p}_n)} \frac{R_{nm}}{\sum_{l=1}^T \sum_{n=1}^V x_{\tau_l}^{nm}} \\ &= \sum_{n=1}^V \left( |\mathcal{O}(\mathbf{p}_n)| - \sum_{m \in \mathcal{O}(\mathbf{p}_n)} \frac{\sum_{l=1}^T \sum_{n=1}^V x_{\tau_l}^{nm} - R_{nm}}{\sum_{l=1}^T \sum_{n=1}^V x_{\tau_l}^{nm}} \right). \end{aligned} \quad (19)$$

Let the potential function  $\Phi(\mathbf{p}'_n, \mathbf{p}_{-n}) - \Phi(\mathbf{p}''_n, \mathbf{p}_{-n}) = \delta_1$ , and the local utility  $U_n(\mathbf{p}'_n, \mathbf{p}_{-n}) - U_n(\mathbf{p}''_n, \mathbf{p}_{-n}) = \delta_2$ , where  $\mathbf{P}' = (\mathbf{p}'_n, \mathbf{p}_{-n})$ ,  $\mathbf{P}'' = (\mathbf{p}''_n, \mathbf{p}_{-n})$ , for all  $n, i \in \mathcal{N}'$ ,  $\mathbf{p}'_n, \mathbf{p}''_n \in \mathcal{P}_n$ ,  $\mathbf{p}_{-n} \in \{\mathcal{P}_i\}_{i \neq n}$ . We can obtain that

$$\begin{aligned} &\Phi(\mathbf{p}'_n, \mathbf{p}_{-n}) \\ &= \sum_{n=1}^V \left( |\mathcal{O}(\mathbf{p}'_n)| - \sum_{m \in \mathcal{O}(\mathbf{p}'_n)} \frac{\sum_{l=1}^T \sum_{n=1}^V x_{\tau_l}^{nm} - R_{nm}}{\sum_{l=1}^T \sum_{n=1}^V x_{\tau_l}^{nm}} \right) \\ &= |\mathcal{O}(\mathbf{p}'_n)| - \sum_{m \in \mathcal{O}(\mathbf{p}'_n)} \frac{\sum_{l=1}^T \sum_{n=1}^V x_{\tau_l}^{nm} - R_{nm}}{\sum_{l=1}^T \sum_{n=1}^V x_{\tau_l}^{nm}} \\ &\quad + \sum_{i \neq n} \left( |\mathcal{O}(\mathbf{p}_i)| - \sum_{m \in \mathcal{O}(\mathbf{p}_i)} \frac{\sum_{l=1}^T \sum_{n=1}^V x_{\tau_l}^{nm} - R_{im}}{\sum_{l=1}^T \sum_{n=1}^V x_{\tau_l}^{nm}} \right) \\ &= |\mathcal{O}(\mathbf{p}'_n)| - \sum_{m \in \mathcal{O}(\mathbf{p}'_n)} \frac{\sum_{l=1}^T \sum_{n=1}^V x_{\tau_l}^{nm} - R_{nm}}{\sum_{l=1}^T \sum_{n=1}^V x_{\tau_l}^{nm}} \\ &\quad - \sum_{i \neq n} \sum_{m \in \mathcal{O}(\mathbf{p}_i) \cap \mathcal{O}(\mathbf{p}'_n)} \frac{\sum_{l=1}^T \sum_{n=1}^V x_{\tau_l}^{nm} - R_{im}}{\sum_{l=1}^T \sum_{n=1}^V x_{\tau_l}^{nm}} \\ &\quad + \sum_{i \neq n} \left( |\mathcal{O}(\mathbf{p}_i)| - \sum_{m \in \{\mathcal{O}(\mathbf{p}_i) \cap \mathcal{O}(\mathbf{p}_z)\}_{z \neq n}} \frac{\sum_{l=1}^T \sum_{n=1}^V x_{\tau_l}^{nm} - R_{im}}{\sum_{l=1}^T \sum_{n=1}^V x_{\tau_l}^{nm}} \right) \\ &= |\mathcal{O}(\mathbf{p}_n)| - \sum_{m \in \mathcal{O}(\mathbf{p}_n)} \left( \sum_{l=1}^T \sum_{n=1}^V x_{\tau_l}^{nm} - \frac{\sum_{n=1}^V R_{nm}}{\sum_{l=1}^T \sum_{n=1}^V x_{\tau_l}^{nm}} \right) + \Delta \\ &= U_n(\mathbf{p}'_n, \mathbf{p}_{-n}) + \Delta, \end{aligned} \quad (20)$$

and similarly,  $\Phi(\mathbf{p}''_n, \mathbf{p}_{-n}) = U_n(\mathbf{p}''_n, \mathbf{p}_{-n}) + \Delta$ , where

$$\Delta = \sum_{i \neq n} \left( |\mathcal{O}(\mathbf{p}_i)| - \sum_{m \in \{\mathcal{O}(\mathbf{p}_i) \cap \mathcal{O}(\mathbf{p}_z)\}_{z \neq n}} \frac{\sum_{l=1}^T \sum_{n=1}^V x_{\tau_l}^{nm} - R_{im}}{\sum_{l=1}^T \sum_{n=1}^V x_{\tau_l}^{nm}} \right). \quad (21)$$

Then, we can get that

$$\Phi(\mathbf{p}'_n, \mathbf{p}_{-n}) - \Phi(\mathbf{p}''_n, \mathbf{p}_{-n}) = U_n(\mathbf{p}'_n, \mathbf{p}_{-n}) - U_n(\mathbf{p}''_n, \mathbf{p}_{-n}). \quad (22)$$

Therefore, it follows that  $\delta_1 = \delta_2$ . According to Lemma 1,  $G$  is an exact potential game. Based on the above, the proof of Theorem 1 is completed.

APPENDIX B  
PROOF OF THEOREM 2

We first represent the problem  $\mathbf{P}_{\text{sub}}$  in a matrix form. Let  $\mathbf{z} = [\mathbf{x}_1; \mathbf{x}_2; \mathbf{x}_3] \in \mathbb{R}^{(I+1)(|\mathcal{V}_u^{\tau_l}| + |\mathcal{E}_u^{\tau_l}|)}$  denote the joint optimization variable, where  $\mathbf{x}_1 = [x_{k,e}^{nm}, \forall e \in \mathcal{V}_u^{\tau_l}] \in \mathbb{R}^{|\mathcal{V}_u^{\tau_l}|}$ ,  $\mathbf{x}_2 = [x_{k,e_{i-1}e_i}^{nm}, \forall e \in \mathcal{V}_u^{\tau_l}, i \in \{1, \dots, I\}] \in \mathbb{R}^{I|\mathcal{V}_u^{\tau_l}|}$  and  $\mathbf{x}_3 = [x_{k,ef}^{nm}, \forall (e, f) \in \mathcal{E}_u^{\tau_l}, i \in \{0, \dots, I\}] \in \mathbb{R}^{(I+1)|\mathcal{E}_u^{\tau_l}|}$ . Note that the notation  $;$  is used to indicate the beginning of a new row. We denote  $\mathbf{B}_1 \in \mathbb{R}^{(I+1)|\mathcal{V}_u^{\tau_l}| \times (I+1)(|\mathcal{V}_u^{\tau_l}| + |\mathcal{E}_u^{\tau_l}|)}$  and  $\mathbf{B}_2 \in \mathbb{R}^{I|\mathcal{V}_u^{\tau_l}| \times (I+1)(|\mathcal{V}_u^{\tau_l}| + |\mathcal{E}_u^{\tau_l}|)}$  as the matrices corresponding

to constraints **C2** and **C6**, respectively. Then the problem  $\mathbf{P}_{\text{sub}}$  can be written in a matrix form as follows,

$$\begin{aligned} \min_{\mathbf{z}} \quad & \mathbf{c}^\top \mathbf{z} \\ \text{s.t.} \quad & \mathbf{B}_1 \mathbf{z} = \mathbf{r}', \\ & \mathbf{B}_2 \mathbf{z} \leq 0, \\ & \mathbf{z} \in \{0, 1\}. \end{aligned} \quad (23)$$

where  $\mathbf{r}'$  is an integral vector having 1, -1 or 0,  $\mathbf{c}$  is the cost vector representing the transmission delay, computation delay and queuing delay. The inner product of  $\mathbf{c}$  and  $\mathbf{z}$  denotes the cost of  $T_k^{nm}$ .

We introduce the non-negative slack variable  $\mathbf{y}_1 \in \mathbb{R}^{(I+1)(|\mathcal{V}_u^{\tau_l}| + |\mathcal{E}_u^{\tau_l}|)}$  to convert the inequality constraint to an equation constraint and relax the variable  $\mathbf{z}$ . Then (23) can be transformed into the following form.

$$\begin{aligned} \min_{\mathbf{z}} \quad & \mathbf{c}^\top \mathbf{z} \\ \text{s.t.} \quad & \mathbf{B}_1 \mathbf{z} = \mathbf{r}', \\ & \mathbf{B}_2 \mathbf{z} + \mathbf{y}_1 = 0, \\ & \mathbf{z} \leq 1, \\ & \mathbf{z}, \mathbf{y}_1 \geq 0. \end{aligned} \quad (24)$$

We now introduce the non-negative slack variable  $\mathbf{y}_2 \in \mathbb{R}^{I|\mathcal{V}_u^{\tau_l}|}$  to transform the inequality constraint  $\mathbf{z} \leq 1$  into the equation constraint. Then the (24) can be written as

$$\begin{aligned} \min_{\mathbf{z}} \quad & \mathbf{c}'^\top \mathbf{x} \\ \text{s.t.} \quad & \mathbf{B} \mathbf{x} = \mathbf{r}, \\ & \mathbf{z}, \mathbf{y}_1, \mathbf{y}_2 \geq 0. \end{aligned} \quad (25)$$

where

$$\mathbf{B} = \begin{bmatrix} \mathbf{B}_1 & 0 & 0 \\ \mathbf{B}_2 & \mathbf{I}_{22} & 0 \\ \mathbf{I}_{31} & 0 & \mathbf{I}_{33} \end{bmatrix}, \mathbf{x} = \begin{bmatrix} \mathbf{z} \\ \mathbf{y}_1 \\ \mathbf{y}_2 \end{bmatrix}, \mathbf{r} = \begin{bmatrix} \mathbf{r}' \\ 0 \\ 1 \end{bmatrix}. \quad (26)$$

Note that  $\mathbf{B} \in \mathbb{R}^{[(3I+2)|\mathcal{V}_u^{\tau_l}| + (I+1)|\mathcal{E}_u^{\tau_l}|] \times [(5I+3)|\mathcal{V}_u^{\tau_l}| + (I+1)|\mathcal{E}_u^{\tau_l}|]}$  and  $\mathbf{c}'$  is the augmented vector of  $\mathbf{c}$  padded with zero to match the dimension of  $\mathbf{x}$ . The matrices  $\mathbf{I}_{ij}$  are identity matrices of appropriate sizes.

According to [24], we can obtain the following definitions:

- 1) The matrix  $\mathbf{B}$  is totally unimodular (TU) if the determinant of each square submatrix is equal to 0, 1 or -1.
- 2) A polyhedron  $\mathcal{P} \subseteq \mathbb{R}^{(5I+3)|\mathcal{V}_u^{\tau_l}| + (I+1)|\mathcal{E}_u^{\tau_l}|}$  is a set of points that satisfy a finite number of linear inequalities, that is,  $\mathcal{P} = \{\mathbf{x} \in \mathbb{R}^{(5I+3)|\mathcal{V}_u^{\tau_l}| + (I+1)|\mathcal{E}_u^{\tau_l}|} : \mathbf{B}\mathbf{x} \leq \mathbf{r}\}$ . Then a nonempty  $\mathcal{P}$  is said to be integral if all of its extreme points are integral. Note that a point in  $\mathcal{P}$  is extreme if it cannot be denoted as a nontrivial convex combination of two distinct points in  $\mathcal{P}$ .

The following lemmas can also be obtained from [24], which are the key to our proof.

**Lemma 2:** If the matrix  $\mathbf{B}$  is TU, then  $\mathcal{P}(\mathbf{r}) = \{\mathbf{x} \in \mathbb{R}^{(5I+3)|\mathcal{V}_u^{\tau_l}| + (I+1)|\mathcal{E}_u^{\tau_l}|} : \mathbf{B}\mathbf{x} \leq \mathbf{r}\}$  is integral for which it is nonempty (it can be also applied to  $\mathcal{P}(\mathbf{r}) = \{\mathbf{x} \in \mathbb{R}^{(5I+3)|\mathcal{V}_u^{\tau_l}| + (I+1)|\mathcal{E}_u^{\tau_l}|} : \mathbf{B}\mathbf{x} = \mathbf{r}\}$ ).

**Lemma 3:** For a  $\mathcal{P} \subseteq \mathbb{R}^{(5I+3)|\mathcal{V}_u^{\tau_l}| + (I+1)|\mathcal{E}_u^{\tau_l}|}$ , a solvable linear programming (i.e.,  $\min\{\mathbf{c}^\top \mathbf{x} : \mathbf{x} \in \mathcal{P}\}$ ) has an integral optimal solution for all  $\mathbf{c} \in \mathbb{R}^{(5I+3)|\mathcal{V}_u^{\tau_l}| + (I+1)|\mathcal{E}_u^{\tau_l}|}$  if and only

if  $\mathcal{P}$  is integral.

**Lemma 4:** For an integral matrix  $B$ , if  $B$  is TU, we can obtain that  $B^\top$  is TU and for each  $\mathcal{W} \subseteq \{1, \dots, (5I+3)|\mathcal{V}_u^{\tau_l}| + (I+1)|\mathcal{E}_u^{\tau_l}|\}$ , there exists a partition  $(\mathcal{W}_1, \mathcal{W}_2)$  of  $\mathcal{W}$  such that  $|\sum_{v \in \mathcal{W}_1} n_{uv} - \sum_{v \in \mathcal{W}_2} n_{uv}| \leq 1$ , where  $u \in \{1, \dots, (3I+2)|\mathcal{V}_u^{\tau_l}| + (I+1)|\mathcal{E}_u^{\tau_l}|\}$ . The above means that for each row, the partitioned sum (called the partitioned row sum) must differ by at most 1. Then we can summarize the above with the following statement: for each  $\mathcal{Q} \subseteq \{1, \dots, (3I+2)|\mathcal{V}_u^{\tau_l}| + (I+1)|\mathcal{E}_u^{\tau_l}|\}$ , there exists a partition  $(\mathcal{Q}_1, \mathcal{Q}_2)$  of  $\mathcal{Q}$  such that  $|\sum_{u \in \mathcal{Q}_1} n_{uv} - \sum_{u \in \mathcal{Q}_2} n_{uv}| \leq 1$ , where  $V \in \{1, \dots, (5I+3)|\mathcal{V}_u^{\tau_l}| + (I+1)|\mathcal{E}_u^{\tau_l}|\}$ .

According to the above, if the constraint matrix is TU, then solving linear programming with the integral vector  $r$  in  $\mathcal{P}$  gives an integral optimal solution. Therefore, we then need to prove the TU of matrix  $B$ .

Before the proof, we give the structure and element distribution of the matrices  $B_1$  and  $B_2$ . Let  $B_1 = [B_{11} \ B_{12} \ B_{13}]$  and  $B_2 = [B_{21} \ B_{22} \ B_{23}]$ . The first  $|\mathcal{V}_u^{\tau_l}|$  columns correspond to the variables  $x_{k,e}^{nm}, \forall e \in \mathcal{V}_u^{\tau_l}$ . The next  $I|\mathcal{V}_u^{\tau_l}|$  columns correspond to the variables  $x_{k,e_{i-1}e_i}^{nm}, \forall e \in \mathcal{V}_u^{\tau_l}, \forall i \in \{1, \dots, I\}$ . The last  $(I+1)|\mathcal{E}_u^{\tau_l}|$  columns correspond to the variables  $x_{k,ef}^{nm}, \forall (e, f) \in \mathcal{E}_u^{\tau_l}, i \in \{0, \dots, I\}$ .

Each row of matrix  $B_1$  corresponds to a flow conservation constraint in **C2** (e.g., for the node  $e \in \mathcal{V}_u^{\tau_l}$ , the first  $I+1$  rows correspond to the flow conservation at replicated nodes  $e_i$  in each layer  $i \in \{1, \dots, I\}$ ). The matrix  $B_{12}$  corresponds to the variables  $x_{k,e_{i-1}e_i}^{nm}, i \in \{1, \dots, I\}$  of cross-layer edges. Constraints are ordered by layer (i.e.,  $e_0, \dots, e_I$ ). Since the edges are between consecutive nodes, for the elements in matrix  $B_{12}$ , start from the first  $I+1$  row and  $I$  column submatrix, where  $u_{cd} = 1, u_{c+1d} = -1, c = \{1, \dots, I\}, d = \{1, \dots, I\}$  and the remaining elements are 0. Then follow the above rules until the  $|\mathcal{V}_u^{\tau_l}|$ -th submatrix, where  $u_{cd} = 1, u_{c+1d} = -1, c = \{(I+1)(|\mathcal{V}_u^{\tau_l}| - 1) + 1, \dots, (I+1)|\mathcal{V}_u^{\tau_l}| - 1\}, d = \{I(|\mathcal{V}_u^{\tau_l}| - 1) + 1, \dots, I|\mathcal{V}_u^{\tau_l}|\}$  and the remaining elements are 0. The rest of the submatrices in matrix  $B_{12}$  are zero matrices. The matrix  $B_{13}$  corresponds to the variables  $x_{k,ef}^{nm}, \forall (e, f) \in \mathcal{E}_u^{\tau_l}, i \in \{0, \dots, I\}$  of intra-layer edges. Specifically, matrix  $B_{13}$  is the  $I+1$ -th diagonal replication of the connection matrix for each node  $e \in \mathcal{V}_u^{\tau_l}$  in the network. Further, the matrix  $B_{11}$  is a zero matrix.

Each row of matrix  $B_2$  corresponds to a constraint in **C6**. In matrix  $B_{21}$ , it is represented at which layer of the graph  $\mathcal{G}^{\tau_l}$  the node  $e \in \mathcal{V}_u^{\tau_l}$  performs the computation, where the elements in the first  $1, \dots, I$  rows of column 1 have a value of -1, the elements in the second  $(I+1), \dots, 2I$  rows of column 2 have a value of -1, and so on, until the elements in the  $I|\mathcal{V}_u^{\tau_l}| - 1 + 1, \dots, I|\mathcal{V}_u^{\tau_l}|$  rows of column  $|\mathcal{V}_u^{\tau_l}|$  have a value of -1. The remaining elements in matrix  $B_{21}$  are 0. Moreover, matrix  $B_{22}$  is an identity matrix and matrix  $B_{23}$  is a zero matrix.

**Lemma 5:** The matrix  $[B_1; B_2]$  is TU.

**Proof:** We denote  $\mathbf{o} \in \mathbb{R}^{(I+1)|\mathcal{V}_u^{\tau_l}|}$  as the vector of row sums from all the columns of matrix  $B_{12}$ . Then we can obtain that  $\mathbf{o}$  has the same numbers of element 1 and element -1. Since

the matrix  $B_{11}$  is a zero matrix and each column of  $B_{13}$  has exactly one element 1 and one element -1, we can obtain that the matrix  $B_1$  satisfies the condition in Lemma 4. Then for the matrix  $B_2$ , all of the row sums satisfy the condition in Lemma 4 since the matrix  $B_{23}$  is a zero matrix. According to the above and Lemma 4, we can obtain that the matrix  $[B_1; B_2]$  is TU.

**Lemma 6:** The matrix  $B$  is TU.

**Proof:** For the first block column of matrix  $B$ , since any column partition of extra identity matrix  $I_{31}$  satisfies the condition in Lemma 4, we can get that the first block column of matrix  $B$  is TU. Then consider adding identity matrices  $I_{22}$  and  $I_{33}$ , since each column in the second and third block columns of matrix  $B$  has only one element 1, we can make the partitioned row sum difference 1 or 0 by adjusting the partition  $(\mathcal{Q}_1, \mathcal{Q}_2)$  (i.e., satisfy the condition in Lemma 4). According to the above analysis, we can get that the matrix  $B$  is TU.

According to Lemma 2, 3, 5 and 6, we can get that the (24) has an integral optimal solution. Then we can solve the routing and computation node selection problem of subshard  $\pi_k^{nm}$  by the following relaxation problem to obtain the integral optimal solution.

$$\begin{aligned} \min_{\mathbf{z}} \quad & \mathbf{c}^\top \mathbf{z} \\ \text{s.t.} \quad & \mathbf{B}_1 \mathbf{z} = \mathbf{r}', \\ & \mathbf{B}_2 \mathbf{z} \leq \mathbf{0}, \\ & \mathbf{0} \leq \mathbf{z} \leq \mathbf{1}. \end{aligned} \quad (27)$$

Accordingly, the proof of Theorem 2 is completed.

## APPENDIX C

### PROOF OF THEOREM 4

In the following proof, we use the service time to represent the sum of the computation time and the transmission time. Recall that we denote  $s \in \{1, \dots, S^{\tau_l}\}$  as the index of the shard arriving in the time slot  $\tau_l$  and  $s_j$  as the  $j$ -th shard that performs the routing and computation node selection policies under the greedy scheme, where  $j \in \{1, \dots, J\}$  and  $J = S^{\tau_l}$ . Let  $W_s^\Gamma$  and  $F_s^\Gamma$  denote the total waiting time and the total service time of shard  $s$  under the policy  $\Gamma$ , respectively. Then we use  $W_s^\Gamma(\text{tran})$  and  $W_s^\Gamma(\text{comp})$  to represent the total transmission waiting time and the total computation waiting time of shard  $s$  under the policy  $\Gamma$ , respectively. Further, we define the total transmission time and the total computation time of shard  $s$  under the policy  $\Gamma$  as  $F_s^\Gamma(\text{tran})$  and  $F_s^\Gamma(\text{comp})$ , respectively.

According to the above definition, we can obtain that the time cost of shard  $s$  under the policy  $\Gamma$  can be denoted as  $W_s^\Gamma + F_s^\Gamma$ . We can also get that  $F_s^\Gamma = F_s^\Gamma(\text{tran}) + F_s^\Gamma(\text{comp})$  and  $W_s^\Gamma = W_s^\Gamma(\text{tran}) + W_s^\Gamma(\text{comp})$ . Then we introduce several policies as follows:

- **Optimal Solution (OS):** Optimal routing and computation node selection policies that achieve the lowest time cost for the shards arriving in the time slot  $\tau_l$ .
- **Minimum Service Time (MST):** Routing and computation node selection policies that achieve the lowest service time.

- Greedy Scheme (GS): Our proposed greedy scheme in Algorithm 3.
- Minimum Queuing Time (MQT): In the time slot  $\tau_l$ , given that  $j-1$  shards have been performed by our proposed greedy algorithm for routing and computation node selection, the  $j$ -th shard is assigned to a node  $e^*$  with minimum computation waiting time. Then, a minimum transmission waiting path is assigned from the source node to  $e^*$  and from  $e^*$  to the destination node. According to the above, transmission only occurs in layers 0 and  $I$  of  $\mathcal{G}^{\tau_l}$  in MQT policy.

Note that after the execution of Algorithm 2, all the subshards in the shard complete the transmission and computation at the same time. In each time slot  $\tau_l$ , due to the greedy scheme perform the routing and computation node selection for the shard with the lowest time cost first, the service time plus waiting time of the last shard in the greedy scheme is the total time cost of the greedy scheme, which can be denoted as  $W_{s_j}^{GS} + F_{s_j}^{GS}$ . Therefore, to analyze the total time cost of the proposed greedy scheme, we just need to derive a bound on  $W_{s_j}^{GS} + F_{s_j}^{GS}$ . In the following, we derive to the bounds of service time and waiting time respectively.

1) *Bounds on the service time*: We define the optimal total time cost in time slot  $\tau_l$  as  $T^{\tau_l*}$ , due to the optimal total time cost  $T^{\tau_l*}$  cannot be smaller than the fastest possible service time of every shard  $s \in \{1, \dots, S^{\tau_l}\}$ , we can obtain that,

$$F_s^{MST} \leq T^{\tau_l*}, s \in \{1, \dots, S^{\tau_l}\}, \quad (28)$$

where  $F_s^{MST}$  denotes the fastest service time of shard  $s$ . Then due to that the total service time of OS cannot be smaller than that of MST and the average busy time of network elements (*i.e.*, computation nodes and transmission links) under OS would not exceed  $T^{\tau_l*}$ , we can get that

$$\frac{1}{|\mathcal{V}_u^{\tau_l}| + |\mathcal{E}_u^{\tau_l}|} \sum_{s=1}^{S^{\tau_l}} F_s^{MST} \leq \frac{1}{|\mathcal{V}_u^{\tau_l}| + |\mathcal{E}_u^{\tau_l}|} \sum_{s=1}^{S^{\tau_l}} F_s^{OS} \leq T^{\tau_l*}. \quad (29)$$

Recall that we denote  $Z_s^H$  and  $Z_s^L$  as the longest and shortest path lengths in hop count between the source node and the destination node, respectively. Further, we define  $Z^H = \max_s Z_s^H$  and  $Z^L = \max_s Z_s^L$ . For the MQT policy, due to the transmission only occurring in layers 0 and  $I$  of  $\mathcal{G}^{\tau_l}$ , the transmission path only needs to find the shortest transmission waiting path in these two layers. Therefore, the transmission time for MQT is at most  $Z^H \max_{k,i} v_k^s / \min_{(e,f)} \omega_{ef}$ , where  $k \in \{1, \dots, K^{nm}\}$  and  $v_k^s$  denotes the size of subshard  $k$  splitted by Algorithm 2 for shard  $s$ . Then the upper bound of transmission time under MQT policy can be denoted as

$$F_s^{MQT}(tran) \leq 2 \frac{Z^H \max_{k,i} v_k^{nm}}{\min_{(e,f)} \omega_{ef}}. \quad (30)$$

Additionally, the transmission time of the MST policy is at least  $Z^L \min_{s,k,i} v_k^s / \max_{(e,f)} \omega_{ef}$ . Then We can obtain the following inequality by combining it with (30).

$$F_s^{MQT}(tran) \leq 2\alpha_{tran} F_s^{MST}(tran), \forall s \in \{1, \dots, S^{\tau_l}\}, \quad (31)$$

where  $\alpha_{tran} = \frac{Z^H \max_{s,k,i} v_k^s \max_{(e,f)} \omega_{ef}}{Z^L \min_{s,k,i} v_k^s \min_{(e,f)} \omega_{ef}}$ . Same as above, if we apply the same argument as above to  $I+1$  layers of  $\mathcal{G}_u^{\tau_l}$

since the greedy scheme is to find a simple path at each layer  $i$ , we can get that

$$F_s^{GS}(tran) \leq (I+1)\alpha_{tran} F_s^{MST}(tran), \forall s \in \{1, \dots, S^{\tau_l}\}. \quad (32)$$

For all policies, since the best case is that all subtasks are computed on the node with the largest computation capacity, and the worst case is that all subtasks are computed on the node with the smallest computation capacity, from which we can obtain that

$$F_s^{\Gamma}(comp) \leq \alpha_{comp} F_s^{MST}(comp), \forall s \in \{1, \dots, S^{\tau_l}\}, \quad (33)$$

where  $\alpha_{comp} = \frac{\max_e \mu_e}{\min_e \mu_e}$ .

For the  $J$ -th shard, according to (31) and (33), we have

$$\begin{aligned} F_{s_j}^{MQT} &= F_{s_j}^{MQT}(tran) + F_{s_j}^{MQT}(comp) \\ &\leq 2\alpha_{tran} F_{s_j}^{MST}(tran) + \alpha_{comp} F_{s_j}^{MST}(comp) \\ &\leq \alpha_F T^{\tau_l*}, \end{aligned} \quad (34)$$

where  $\alpha_F = \max(2\alpha_{tran}, \alpha_{comp})$ .

2) *Bounds on the waiting time*: Since each satellite in LEO satellite networks has  $A$  antennas (*i.e.*,  $\mathcal{G}_u^{\tau_l}$  is  $A$ -edge-connected). Then due to that the MQT policy performs routing and computation node selection prior to shard  $s_j$ , the remaining  $J-1$  shards have already undergone this process using the GS policy. According to the above, we can obtain that there are  $A$  disjoint paths between any pair of nodes. Then we can get that for the transmission path from the source node to  $e^*$ , the average transmission waiting time along each of  $A$  disjoint paths is at most  $\frac{1}{A} \sum_{j=1}^{J-1} F_{s_j}^{GS}(tran)$ . Since the MQT policy is to choose the path with the minimum transmission waiting time, applying the same argument above to the transmission path from  $e^*$  to the destination node and combining it with (32), we can obtain that

$$\begin{aligned} W_{s_j}^{MQT}(tran) &\leq \frac{2}{A} \sum_{j=1}^{J-1} F_{s_j}^{GS}(tran) \\ &\leq \frac{2(I+1)\alpha_{tran}}{A} \sum_{j=1}^{J-1} F_{s_j}^{MST}(tran). \end{aligned} \quad (35)$$

In addition, since the MQT policy selects the computation node with the minimum waiting time, the computation waiting time of the MQT policy will not be larger than the average node waiting time, and combining this with (33), we can get

$$\begin{aligned} W_{s_j}^{MQT}(comp) &\leq \frac{1}{|\mathcal{V}_u^{\tau_l}|} \sum_{j=1}^{J-1} F_{s_j}^{GS}(comp) \\ &\leq \frac{\alpha_{comp}}{|\mathcal{V}_u^{\tau_l}|} \sum_{j=1}^{J-1} F_{s_j}^{MST}(comp). \end{aligned} \quad (36)$$

For the last decision of shard  $J$ , comparing the MQT policy and the GS policy, since the MQT policy is only based on the minimum waiting time for routing and computation node selection, while the GS policy considers the minimization of the waiting time plus the service time, we can get that

$$F_{s_j}^{GS} + W_{s_j}^{GS} \leq F_{s_j}^{MQT} + W_{s_j}^{MQT}. \quad (37)$$

According to (35) and (36), we have

$$W_{s_j}^{MQT} = W_{s_j}^{MQT}(tran) + W_{s_j}^{MQT}(comp)$$



$$\leq \alpha_W \sum_{j=1}^{J-1} F_{s_j}^{MST}, \quad (38)$$

where  $\alpha_W = \max(\frac{2(I+1)\alpha_{tran}}{A}, \frac{\alpha_{comp}}{|\mathcal{V}_u^{\tau_l}|})$ .

Combining (28), (29), (34) and (38), we can get that

$$\begin{aligned} & F_{s_J}^{MQT} + W_{s_J}^{MQT} \\ & \leq \alpha_F F_{s_j}^{MST} + \alpha_W \sum_{j=1}^{J-1} F_{s_j}^{MST} \\ & \leq \alpha' \{ F_{s_j}^{MST} + \frac{1}{|\mathcal{V}_u^{\tau_l}| + |\mathcal{E}_u^{\tau_l}|} \sum_{j=1}^{J-1} F_s^{MST} \} \\ & = \alpha' \{ (1 - \frac{1}{|\mathcal{V}_u^{\tau_l}| + |\mathcal{E}_u^{\tau_l}|}) F_{s_j}^{MST} + \frac{1}{|\mathcal{V}_u^{\tau_l}| + |\mathcal{E}_u^{\tau_l}|} \sum_{j=1}^J F_s^{MST} \} \\ & \leq \alpha' (2 - \frac{1}{|\mathcal{V}_u^{\tau_l}| + |\mathcal{E}_u^{\tau_l}|}) T^{\tau_l*}, \end{aligned} \quad (39)$$

where  $\alpha' = \max(\alpha_F, \alpha_W(|\mathcal{V}_u^{\tau_l}| + |\mathcal{E}_u^{\tau_l}|))$ . Finally, according to (37), we can get that

$$F_{s_J}^{GS} + W_{s_J}^{GS} \leq \alpha T^{\tau_l*}, \quad (40)$$

where

$$\begin{aligned} \alpha = & \left( 2 - \frac{1}{|\mathcal{V}_u^{\tau_l}| + |\mathcal{E}_u^{\tau_l}|} \right) \max \left\{ \left( 1 + \frac{|\mathcal{E}_u^{\tau_l}|}{|\mathcal{V}_u^{\tau_l}|} \right) \frac{\max_e \mu_e}{\min_e \mu_e}, \right. \\ & \left. \left( 2, \frac{2(I+1)(|\mathcal{V}_u^{\tau_l}| + |\mathcal{E}_u^{\tau_l}|)}{A} \right) \frac{Z^H \max_{s,k,i} v_k^s \max_{(e,f)} \omega_{ef}}{Z^L \min_{s,k,i} v_k^s \min_{(e,f)} \omega_{ef}} \right\}. \end{aligned} \quad (41)$$

Accordingly, the proof of Theorem 4 is completed.